

RT1021-MicroPython

固件接口说明

目录

目录	1
1. 前言	4
2. RT1021-MicroPython 核心板连接与脱机启动	5
2.1. 开源 MicroPython 开发工具	5
2.2. RT1021-MicroPython 核心板连接电脑	7
2.3. Thonny 连接 RT1021-MicroPython 核心板	8
2.4. RT1021-MicroPython 核心板启动顺序说明	10
2.4.1. RT1021-144P 芯片 BTB 连接器接口核心板脱机启动	11
2.4.2. RT1021-144P 芯片 2.54 排针接口核心板脱机启动	11
2.4.3. RT1021-100P 芯片 2.54 排针接口核心板脱机启动	12
2.5. 使用 Thonny 进行文件操作	13
2.6. 注意事项与常见问题	14
3. RT1021-MicroPython 核心板固件说明	15
3.1. 底层接口 machine 库	15
3.1.1. Pin 子模块	15
3.1.2. ADC 子模块	16
3.1.3. PWM 子模块	17
3.1.4. UART 子模块	17
3.1.5. SPI 子模块	19

3.1.6. IIC 子模块	20
3.2. NXP 的 smartcar 库	22
3.2.1. ticker 子模块.....	22
3.2.2. ADC_Group 子模块.....	23
3.2.3. encoder 子模块	24
3.3. NXP 的 display 库.....	25
3.4. 逐飞科技的 seekfree 库	28
3.4.1. MOTOR_CONTROLLER 子模块	28
3.4.2. BLDC_CONTROLLER 子模块	30
3.4.3. KEY_HANDLER 子模块.....	31
3.4.4. IMU660/963RX 子模块.....	33
3.4.5. DL1X 子模块	35
3.4.6. TSL1401 子模块	36
3.4.7. 传感器子模块与 ticker 的 caputer_list 关联采集说明.....	38
3.4.8. WIRELESS_UART 子模块	40
3.4.9. WIFI_SPI 子模块	42
3.4.10. IPS200PRO 子模块	43
4. 多文件调用与类定义.....	59
4.1. 多文件加载	59
4.2. 多文件变量作用域.....	60
4.3. 类的应用	61

5. 文档版本 63

1.前言

本文档的主要作用是帮助各位快速入门 RT1021-MicroPython 开发,主要内容分三个部分。

第一个部分: 环境与硬件连接说明, 内容为环境安装与介绍, 硬件检查与连接, RT1021-MicroPython 核心板硬件启动流程以及注意事项;

第二个部分: 固件接口说明, 内容为核心板固件自带的接口库说明, 用于配合例程学习如何使用对应的接口与传感器驱动;

第三个部分: 关于多文件的加载与引用问题, 以及 Class 的使用示例等。

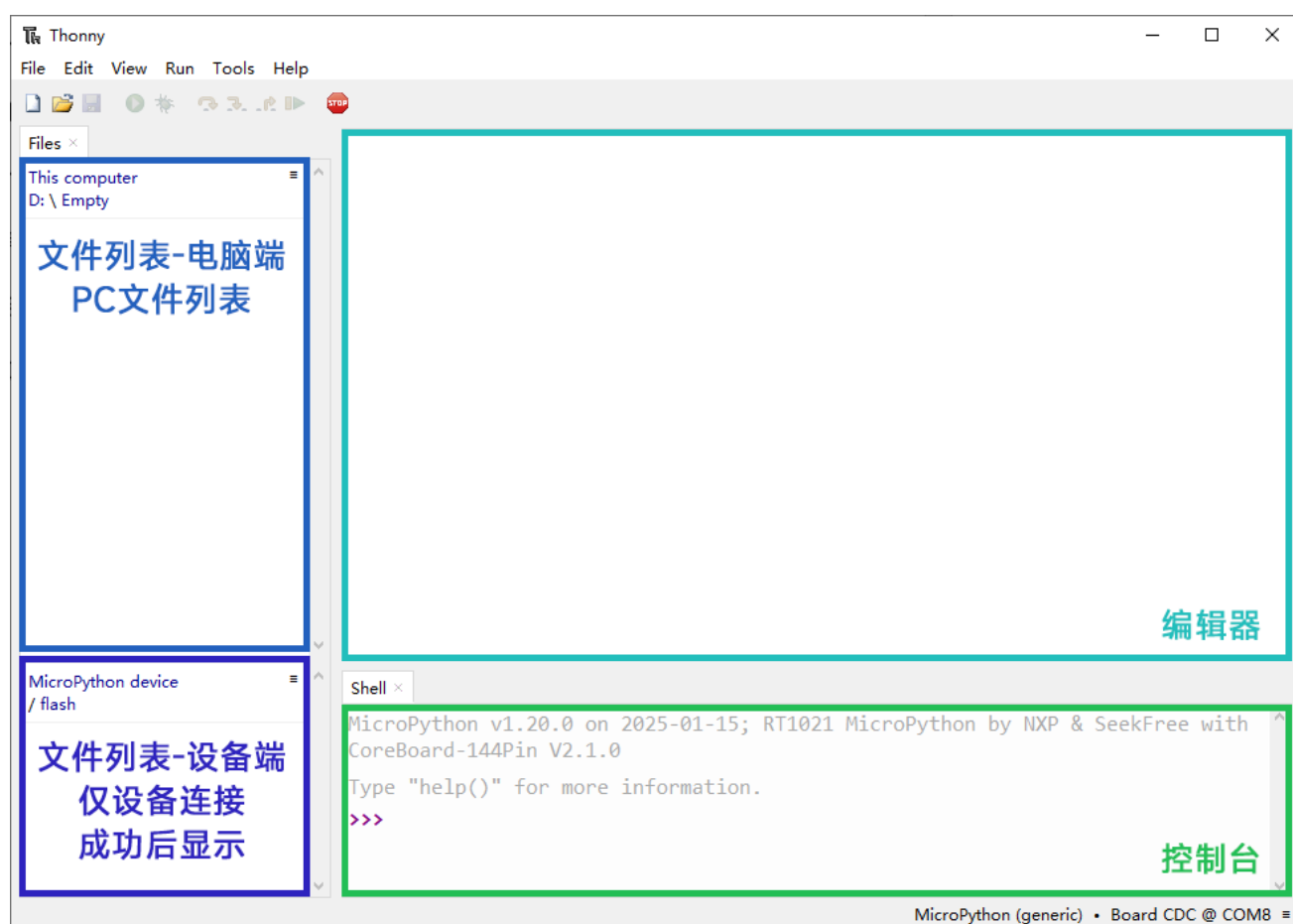
文档并不能直接详尽介绍如何使用 RT1021-MicroPython 核心板完成整个开发流程, 只能作为入门指导和使用手册, 如有良好的修改意见也欢迎各位提出建议。

2.RT1021-MicroPython 核心板连接与脱机启动

2.1.开源 MicroPython 开发工具

进行 MicroPython 开发需要一个代码编辑器、支持 MicroPython 文件协议的文件管理器和一个 REPL 控制台。如果各位有自己熟悉的环境，可以跳过这一个小节。

可以使用开源软件——Thonny，它自带文件系统支持和基于串口连接的 REPL 控制台：

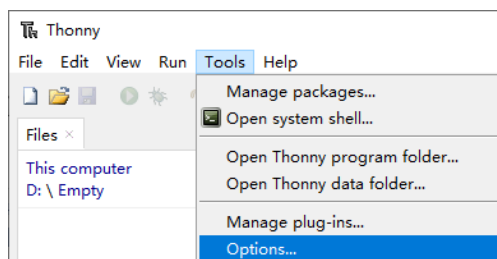


Thonny 软件下载链接：<https://github.com/thonny/thonny/releases>

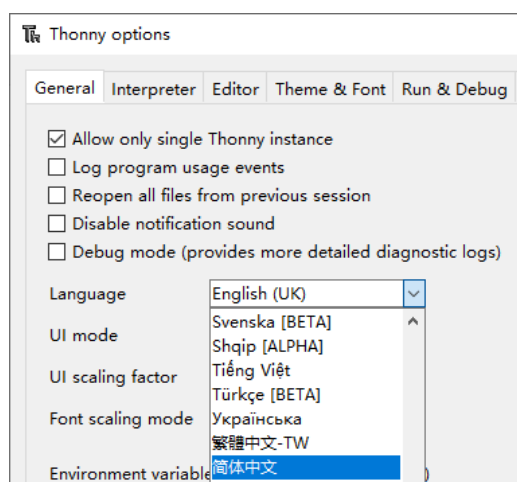
需要使用到一些“上网技巧”访问 GitHub，建议至少使用 V4.1.0 及以上版本的 Thonny，否则与本说明书中使用的版本的界面、功能可能会有较大的差异。

Thonny 源码开源链接：<https://github.com/thonny/thonny>

可以通过工具栏的 Tools->Options...打开设置：

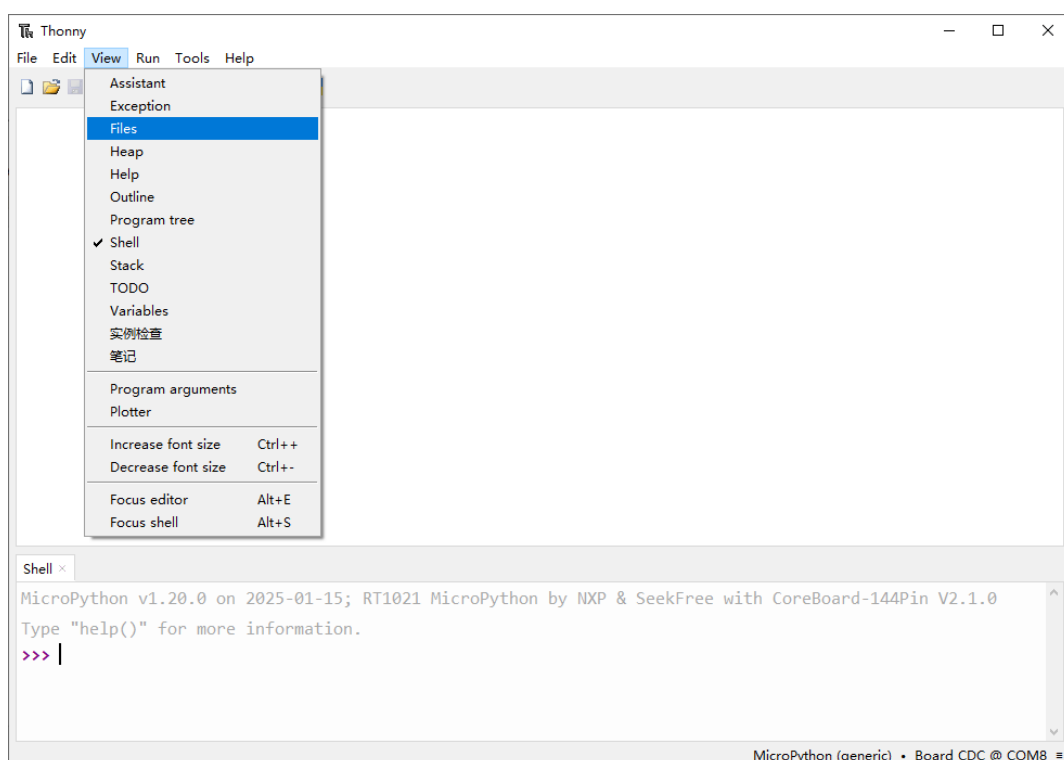


然后在 General 选项卡下的 Language 下拉框选择简体中文，将软件设置为中文模式：



设置完成后需要重启软件生效。

如果界面没有显示文件管理器，可以通过工具栏 View->Files 打开文件管理视图。



2.2.RT1021-MicroPython 核心板连接电脑

使用前，请务必先检查核心板是否正常，**确保没有水滴、金属丝落在核心板上导致短路！**

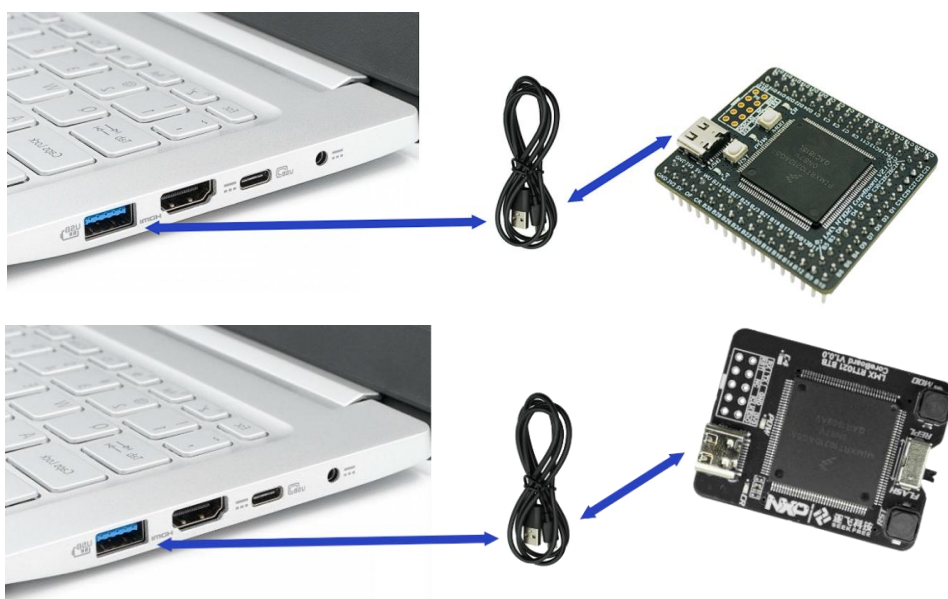
并且尽量做好绝缘防护（可以给核心板上表面裸露金属部分贴绝缘胶带进行保护）。如果是秋冬季，**请尽量做好防静电措施！避免对板子放电导致静电击穿损坏！**

任何电路板都不可以直接放置在金属桌面、笔记本电脑的金属面板上！

任何电路板都不可以直接放置在金属桌面、笔记本电脑的金属面板上！

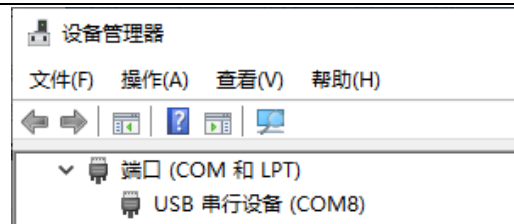
任何电路板都不可以直接放置在金属桌面、笔记本电脑的金属面板上！

确保以上注意事项都符合，就可以使用 Type-C 数据线（**确保使用的是数据线，而不是单纯的充电线！**）将 RT1021-MicroPython 核心板连接到 PC 的 USB 口：

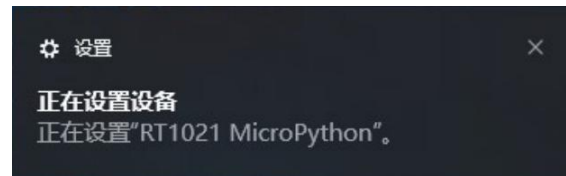


部分笔记本电脑可能没有足够多的 USB 接口而使用 USB 拓展坞，需要注意**有的拓展坞由于版本较老、芯片落后而导致无法正常枚举设备**，如果遇到这类问题，请自行检查并确保 USB 拓展坞能够正常工作。

当连接正常时，系统会有 USB 设备插入的提示音，同时在设备管理器中会新增一个串口设备，在说明书使用的测试 PC 上它的 COM 号是“COM8”，在不同电脑上其编号会有差异：



过一段时间，设备会完成枚举并识别具体信息，可能会在右下角弹出设备提示：



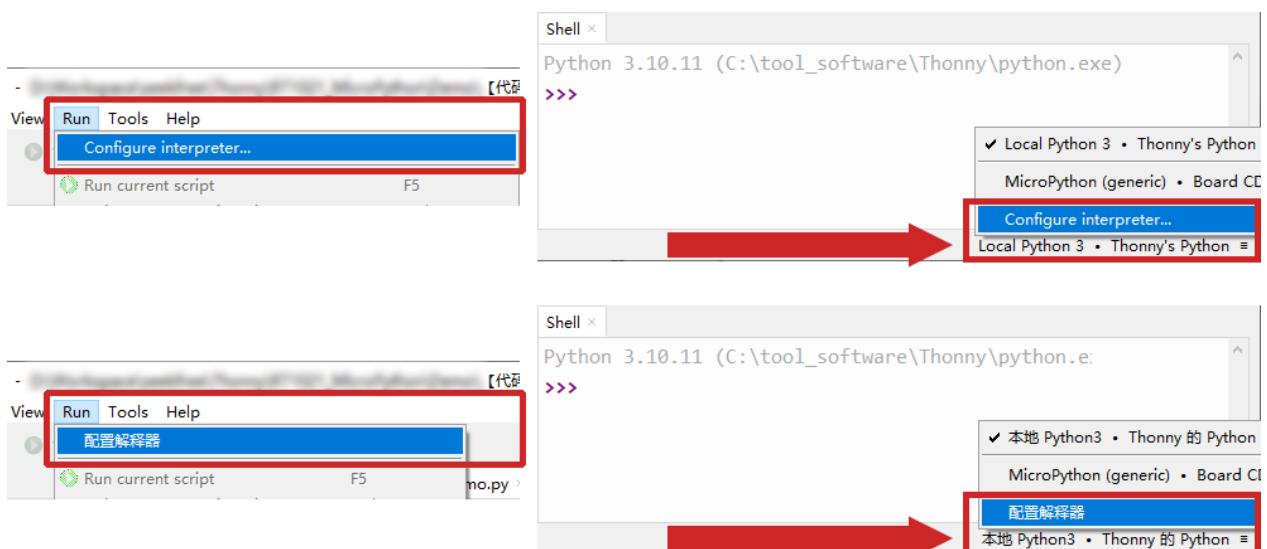
或者可以打开系统设置，找到蓝牙与其他设备，可以在其他设备下找到它：



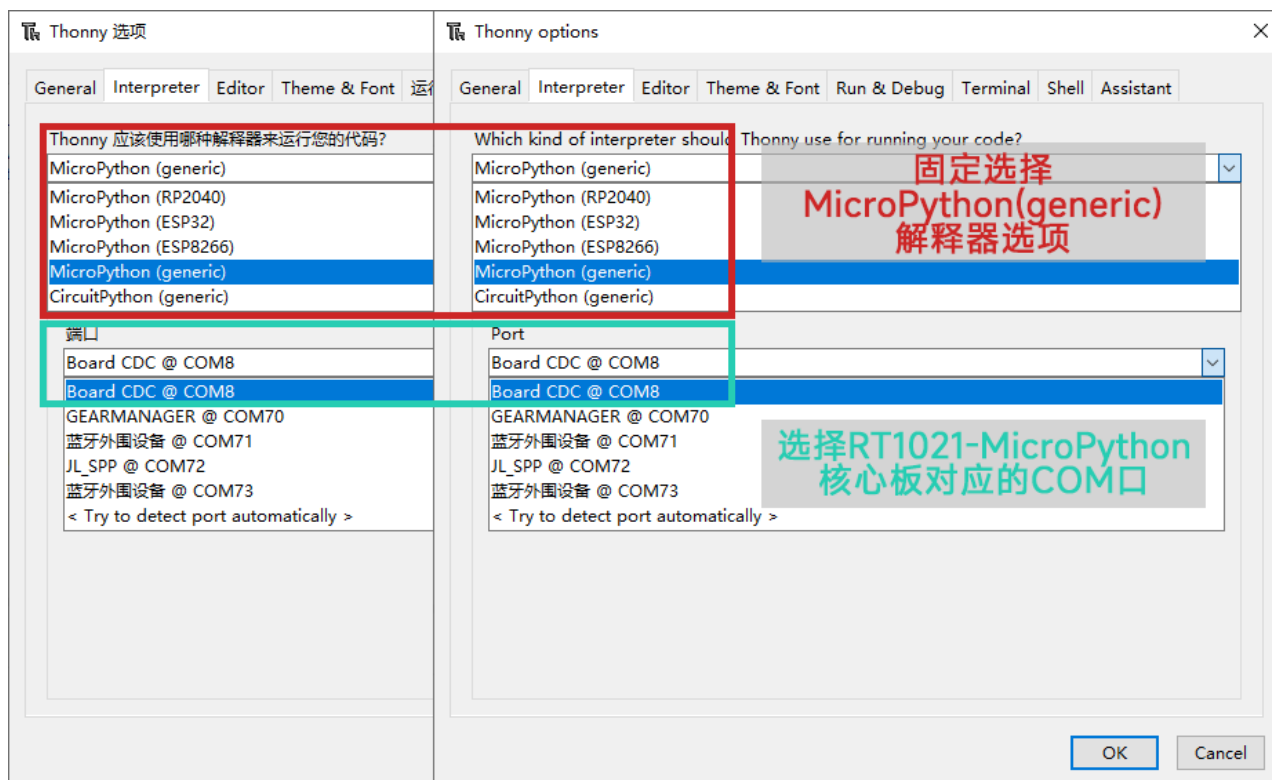
能够识别到 COM 号并显示在其他设备中，就证明设备连接正常。

2.3.Thonny 连接 RT1021-MicroPython 核心板

打开 Thonny，打开菜单栏的“Run->Configure Interpreter”（英文模式）/“运行->解释器配置”（中文模式）选项，或者点击窗口右下角的解释器切换按键：



在 Interpreter 选项卡，选择 MicroPython(generic)解释器：



在 RT1021-MicroPython 核心板单独连接电脑，**不连接任何传感器、不插在学习板或者自己主板上**时, 并且 RT1021-MicroPython 核心板中**并未保存任何 Python 文件**情况下, Thonny 将通过对应的 COM 口连接到 RT1021-MicroPython 核心板并在控制台输出固件信息：

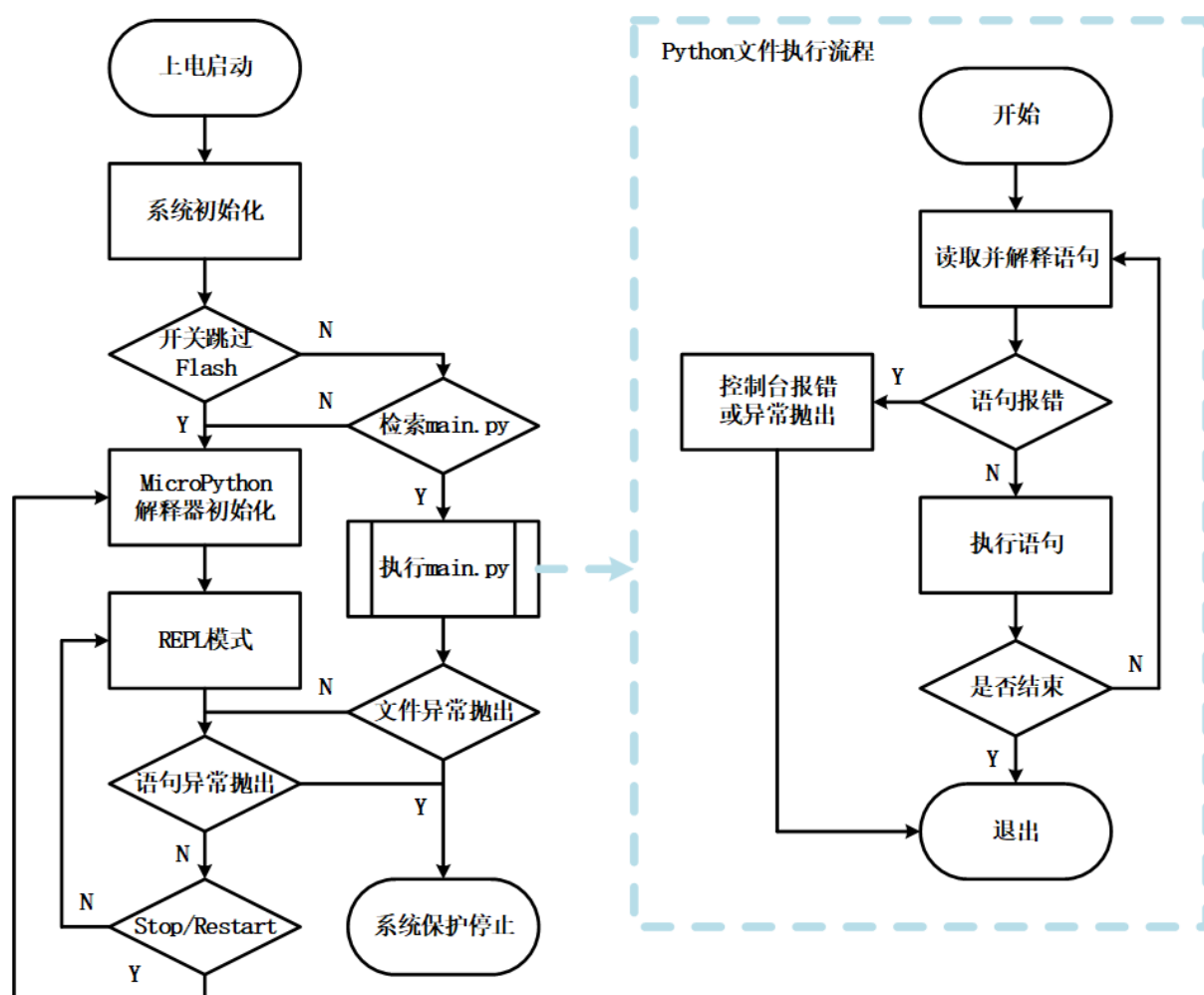


2.4.RT1021-MicroPython 核心板启动顺序说明

由于核心板没有提供额外的 SD 卡插槽，也不支持 USB 连接作为 U 盘进行文件读写，而是使用核心板自带的 Flash 芯片进行文件存储。

但板子的 USB 只作为通信使用，并没有 MSC 协议，所以它只能通过 MicroPython 的文件管理协议来进行文件系统操作，使得它必须依赖 Thonny 这类工具才能进行文件系统操作。

板子上电后 MicroPython 设备会自动检索文件系统中是否存 main.py 文件：



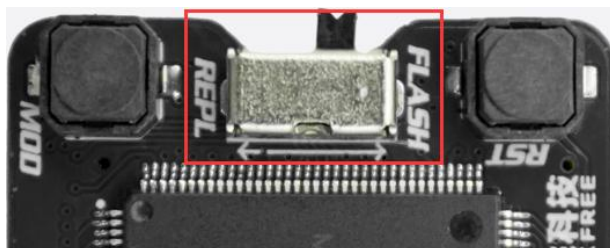
所有核心板均绑定一个 IO 用于脱机启动操作, 所以不论学习板还是主板需要预留这个 IO!

所有核心板均绑定一个 IO 用于脱机启动操作, 所以不论学习板还是主板需要预留这个 IO!

所有核心板均绑定一个 IO 用于脱机启动操作, 所以不论学习板还是主板需要预留这个 IO!

2.4.1.RT1021-144P 芯片 BTB 连接器接口核心板脱机启动

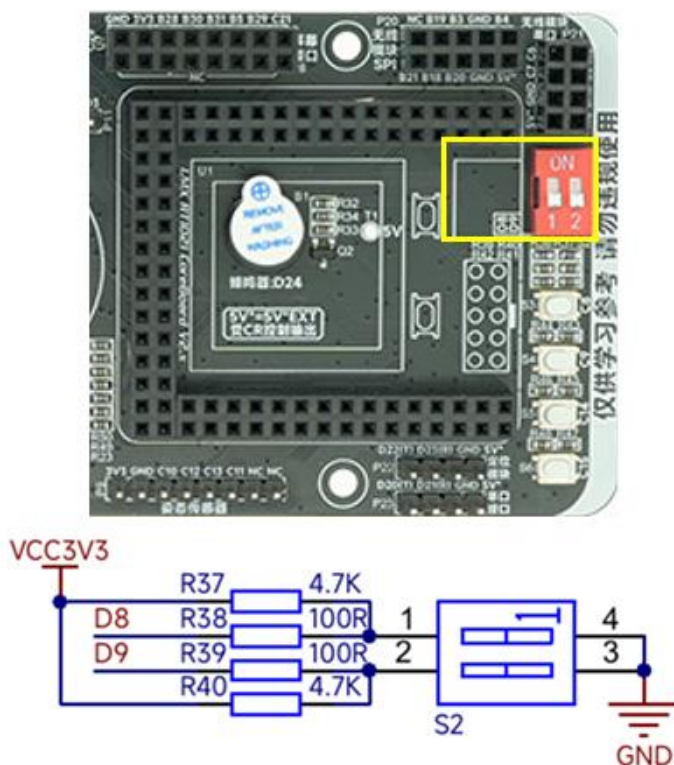
固件绑定 B9 引脚，也就是核心板上的启动模式开关，用来进行脱机启动操作：



将文件以 main.py 保存到 MicroPython 设备中，然后将核心板上的启动模式选择开关拨动到“FLASH”一侧，然后复位一次核心板，在上电后 MicroPython 设备会自动运行 main.py 文件。不需要脱机启动时，将开关拨动到“REPL”一侧复位即可。

2.4.2.RT1021-144P 芯片 2.54 排针接口核心板脱机启动

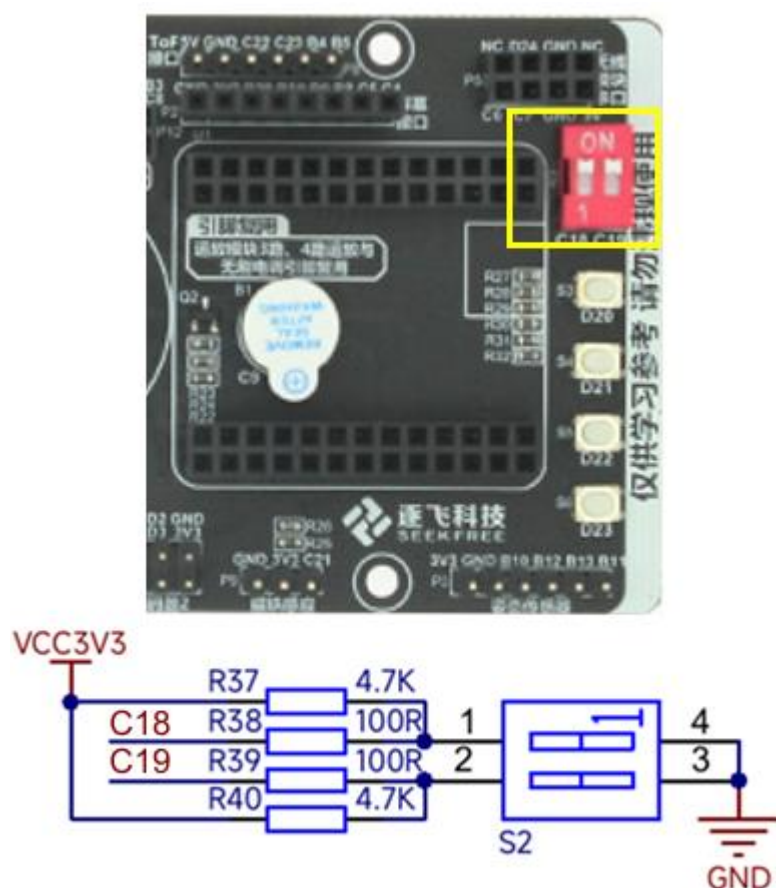
固件绑定 D8 引脚，用来进行脱机启动操作，由于并不是核心板上预留，因此学习板、主板上必须将 D8 连接一个开关：



将文件以 main.py 保存到 MicroPython 设备中，然后将 D8 对应的启动模式选择开关拨动到 GND 低电平一侧(学习板上为 ON 一侧)，然后复位一次核心板，在上电后 MicroPython 设备会自动运行 main.py 文件。不需要脱机启动时，将开关拨动到“REPL”一侧复位即可。

2.4.3.RT1021-100P 芯片 2.54 排针接口核心板脱机启动

固件绑定 C18 引脚，用来进行脱机启动操作，由于并不是核心板上预留，因此学习板、主板上必须将 C18 连接一个开关：



将文件以 main.py 保存到 MicroPython 设备中，然后将 D8 对应的启动模式选择开关拨动到 GND 低电平一侧(学习板上为 ON 一侧)，然后复位一次核心板，在上电后 MicroPython 设备会自动运行 main.py 文件。不需要脱机启动时，将开关拨动到“REPL”一侧复位即可。

2.5.使用 Thonny 进行文件操作

进行文件系统操作时，请务必确认：

使用学习板或主板保证供电稳定，否则掉电导致文件系统异常无法启动、损坏文件。

使用学习板或主板保证供电稳定，否则掉电导致文件系统异常无法启动、损坏文件。

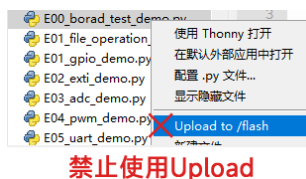
使用学习板或主板保证供电稳定，否则掉电导致文件系统异常无法启动、损坏文件。

通过 Thonny 连接板子，再打开资料附带的任意例程文件，选择工具栏 File->save as...

在弹窗中选择 MicroPython device 将 boot 文件保存：



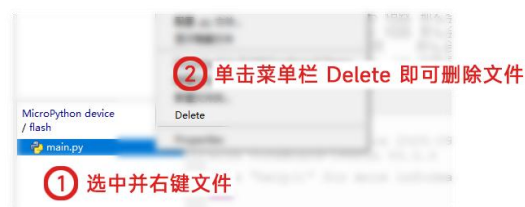
需要注意的是，**禁止使用 Upload 操作**，因为 Upload to /flash 操作支持并不完善，可能会导致文件系统操作超时，此时关闭窗口可能会导致文件错误！



修改文件后直接使用 Ctrl + S 或者单击工具栏的保存按钮即可：



删除文件只需要右键对应的文件，在弹出的菜单中选择对应的选项即可：



2.6.注意事项与常见问题

请务必在使用学习板或主板供电的情况下，再进行文件保存、修改、删除的操作！

请务必在使用学习板或主板供电的情况下，再进行文件保存、修改、删除的操作！

请务必在使用学习板或主板供电的情况下，再进行文件保存、修改、删除的操作！

这是为了保证在文件操作过程中不会因为 Type-C 松动导致断电，从而使得 Flash 芯片操作异常，导致文件系统异常，最终使得脱机文件无法正常启动、使用。

问题一：代码运行显示“**MemoryError: memory allocation failed, allocating **** bytes.**”

需要检查代码中是否存在冗杂的变量、对象申请，导致内存泄露无法运行。一般常见于 Python 代码有问题，无法在脱机运行后再次连接 Thonny 运行。需要自己优化代码解决，通常来说，可以将全局变量通过 class 封装，通过传递对象的方式使用，避免出现多文件作用域问题导致的内存泄露。

问题二：使用 Thonny 手动运行没有发现问题，可以正常执行程序，但脱机不执行文件。

那么尝试拨码开关跳过执行，然后复位连接 Thonny，再运行文件，此时可能会看到报错信息，这类情况一般是代码写的有问题，类或变量调用不正确导致的，只会在第一次运行报错，而第二次运行就正常。

3.RT1021-MicroPython 核心板固件说明

固件总共分成三个部分：NXP 实现的 MicroPython 底层接口 machine 库、NXP 为智能车高实时性应用编写的 smartcar 和 display 库、逐飞科技为智能车应用添加的传感器库。

3.1.底层接口 machine 库

3.1.1.Pin 子模块

GPIO 与外部中断接口，参数与原生略有差异：

```
# -----
# 构造接口 是标准 MicroPython 的 machine.Pin 模块
# Pin_obj = Pin(pin, mode, pull = Pin.PULL_UP_47K, value = 1, drive = Pin.DRIVE_OFF)
#     pin    引脚名称 | 必要参数 引脚名称 本固件以核心板上引脚编号为准
#     mode    引脚模式 | 必要参数 对应引脚工作状态 Pin.x, x = (IN, OUT, OPEN_DRAIN)
#     pul     上拉下拉 | 可选参数 Pin.x, x = (PULL_UP, PULL_UP_47K, PULL_UP_22K, PULL_DOWN, PULL_HOLD)
#     value   初始电平 | 可选参数 关键字参数 可以输入 (0, 1) 或 (False, True) 将端口电平设置为对应 bool 值
#     drive   内阻模式 | 可选参数 关键字参数 Pin.x, x = (PIN_DRIVE_OFF, PIN_DRIVE_0, ..., PIN_DRIVE_6)
#     return 返回内容 | 正常情况下返回对应 Pin 的对象
# -----
from machine import Pin
led = Pin('C4', Pin.OUT, value = True)
key = Pin('D8', Pin.IN, pull = Pin.PULL_UP_47K)
# -----
# 端口电平置位
# Pin.on()
# -----
led.on()
# -----
# 端口电平复位
# Pin.off()
# -----
led.off()
# -----
# 端口电平输出低电平
# Pin.low()
# -----
led.low()
# -----
# 端口电平输出高电平
# Pin.high()
# -----
led.high()
# -----
# 端口电平翻转
# Pin.toggle()
# -----
```



```
led.toggle()
# -----
# 传入参数 level 则将端口电平设置为对应 bool 值
# Pin.value(level)
# level = Pin.value()
# level 电平状态 | 必要参数 可以输入 (0, 1) 或 (False, True) 将端口电平设置为对应 bool 值
# return 返回内容 | 当不输入 level 参数时 返回当前引脚状态 否则无返回内容
# -----
led.value(False)
key_state = key.value()
# -----
# 配置 Pin 的中断 也就是外部中断 EXTI
# Pin.irq(handler, trigger, hard)
# handler 回调函数 | 必要参数 触发后对应的回调函数 python 函数
# trigger 触发模式 | 必要参数 可用值为 Pin.x, x = (IRQ_RISING, IRQ_FALLING)
# hard 应用模式 | 可选参数 可用值为 False True
# -----
def hall_handler (pin_obj):
    global key_state # global 是全局变量修饰 说明这里使用的是一个全局变量 否则函数会新建一个临时变量
    key_state = pin_obj.value()
key.irq(hall_handler, Pin.IRQ_FALLING, False)
```

3.1.2.ADC 子模块

ADC 模块，与原生的 MicroPython 的 ADC 模块基本一致，不过传入的是 Pin 类的对象，或者引脚的名称编号，这里以核心板上标注的引脚编号为准。

```
# -----
# 构造接口 是标准 MicroPython 的 machine.ADC 模块
# ADC_obj = ADC(pin)
# pin 引脚名称 | 必要参数 引脚名称 本固件以核心板上引脚编号为准
# return 返回内容 | 正常情况下返回对应 ADC 通道的对象
# -----
from machine import ADC
power_adc = ADC('B27')
# -----
# 读取当前端口的 ADC 转换值
# ADC.read_u16()
# return 返回内容 | ADC 转换数值 数据返回范围是 [0, 65535]
# -----
power_adc_value = power_adc.read_u16()
```

需要注意的是，引脚可能同时支持 ADC1 和 ADC2 模块的通道输入，调用 ADC 的构造函数时，会自动适配到 ADC1 模块上。

3.1.3.PWM 子模块

PWM 模块，基本兼容 MicroPython 的 PWM 模块：

```
# -----  
# 构造接口 是标准 MicroPython 的 machine.PWM 模块  
# PWM_obj = PWM(pin, freq, [duty])  
#     pin    引脚名称    | 必要参数 对应核心板上有 PWM 功能的引脚  
#     freq   信号频率    | 必要参数  
#     duty   占空比值    | 必要参数 关键字输入 范围 [1, 65535]  
#     return 返回内容    | 正常情况下返回对应 PWM 通道的对象  
# -----  
from machine import PWM  
pwm = PWM("B26", 10 * 1000, duty_u16 = 32768)  
# -----  
# 占空比值接口  
# PWM.duty_u16(duty)  
# duty = PWM.duty_u16()  
#     duty   占空比值    | 可选参数 输入 duty 则更新占空比设置 范围 [1, 65535]  
#     return 返回内容    | 当不输入 duty 参数时 返回当前占空比值 否则无返回内容  
# -----  
pwm.duty_u16(16384)  
print(pwm.duty_u16())  
# -----  
# 信号频率接口  
# PWM.freq(freq)  
# freq = PWM.freq()  
#     freq   信号频率    | 可选参数 输入 freq 则更新频率设置  
#     return 返回内容    | 当不输入 freq 参数时 返回当前信号频率 否则无返回内容  
# -----  
pwm.freq(300)  
print(pwm.freq())
```

这个接口通常用来控制舵机，或者单个的 PWM 引脚，因为它只能使用单个通道，或者把一对硬件互补引脚初始化成互补输出通道。

也不能单独初始化同个子模块下的 A、B 通道无法单独控制占空比，如果单独初始化同一个子模块的 A、B 通道，他们占空比不能独立控制。

3.1.4.UART 子模块

兼容原生 MicroPython 的 UART 的模块。

```
# -----  
# 构造接口 标准 MicroPython 的 machine.UART 模块  
# UART_obj = UART(id)
```

```
# id      接口编号 | 必要参数 本固件支持 (0, 1, 2, 3, 5, 7) 总共 6 个 UART 模块
# return 返回内容 | 正常情况下返回对应 UART 的对象
# -----
from machine import UART
uart6 = UART(5)
# -----
# 串口参数设置
# UART.init(baudrate = 9600, bits = 8, parity = None, stop = 1, ...)
# baudrate  串口速率 | 必要参数 默认 9600
# bits      数据位数 | 可选参数 默认 8 bits 数据位
# parity    校验位数 | 可选参数 默认 None (None, 0, 1) 无校验, 偶校验, 奇校验
# stop      停止位数 | 可选参数 默认 1 bit 停止位
# -----
uart6.init(460800)
# -----
# 将 bufffer 内容通过 UART 发送
# UART.write(bufffer)
# bufffer    数据缓冲 | 必要参数 存放发送数据的缓冲区
# -----
uart6.write("Hello world.\r\n")
# -----
# 查询 UART 是否有数据可读 返回可读取长度
# lenght = UART.any()
# return    返回内容 | 返回缓冲区内数据长度 字节单位
# -----
buf_len = uart6.any()
# -----
# 读取 lenght 字节到 bufffer
# bufffer = UART.read(lenght)
# length    读取长度 | 必要参数 准备读取的数据长度
# return    返回内容 | 返回读取到的内容 为字节数组形式
# -----
buf = uart6.read(buf_len)
```

各版本的资源有所差异，请对照下表：

Board Type		144P-BTB		144P-2.54		100P-2.54	
HW-UART	Logical	TX	RX	TX	RX	TX	RX
LPUART1	id = 0	B6	B7	B6	B7	B6	B7
LPUART2	id = 1	C22	C23	C22	C23	C22	C23
LPUART3	id = 2	C6	C7	C6	C7	C6	C7
LPUART4	id = 3	D0	D1	D0	D1	B26	B27
LPUART5	id = 4			B10	B11	B10	B11
LPUART6	id = 5	D20	D21	D20	D21	D20	D21
LPUART7	id = 6			D17	D18	D2	D3
LPUART8	id = 7	D22	D23	D22	D23	D22	D23

需要注意的是 RT1021-144P-BTB 核心板 LPUART5 / LPUART7 不能使用。

3.1.5.SPI 子模块

SPI 模块，基本兼容 MicroPython 的 SPI 模块：

```
# -----
# 构造接口 标准 MicroPython 的 machine.SPI 模块
# SPI_obj = SPI(id)
# id      接口编号 | 必要参数 本固件支持 [1, 3] 总共 3 个 SPI 模块
# return 返回内容 | 正常情况下返回对应 SPI 的对象
# -----
from machine import SPI
spi2 = SPI(1)
# -----
# SPI 参数设置 参数说明
# SPI.init(baudrate = 1000000, polarity = 0, phase = 0)
# baudrate  传输速率 | 默认 1000000 1Mbps
# polarity  电平极性 | 可选参数 默认 0 {0 - 时钟空闲时低电平, 1 - 时钟空闲时高电平}
# phase     时钟相位 | 可选参数 默认 0 {0 - 第一个时钟沿采样数据, 1 - 第二个时钟沿采样数据}
# -----
spi2.init(baudrate = 1000000, polarity = 0, phase = 0)
# -----
# 读取指定的字节数 同时连续发送指定的单个字节 默认发送 0x00
# rx_buff = spi.read(lenght, wite = 0)
# length   读取长度 | 必要参数 准备读取的数据长度
# wite     发送数据 | 可选参数 默认为 0x00
# return   返回内容 | 返回读取到的内容 为字节数组形式
# -----
rx_byte = spi2.read(1)
# -----
# 读取 rx_buff 长度数据 同时连续发送指定的单个字节 默认发送 0x00
# spi.readinto(rx_buff, wite = 0)
# rx_buff   数据缓冲 | 必要参数 存放读取数据的缓冲区
# wite     发送数据 | 可选参数 默认为 0x00
# -----
spi2.readinto(rx_buff)
# -----
# 发送 tx_buff 长度数据
# spi.write(tx_buff)
# tx_buff   数据缓冲 | 必要参数 存放发送数据的缓冲区
# -----
spi2.write(tx_buff)
# -----
# 发送 tx_buff 长度数据 同时读取数据到 rx_buff 这两个缓冲区必须一样长
# spi.write_readinto(tx_buff, rx_buff)
# rx_buff   数据缓冲 | 必要参数 存放读取数据的缓冲区
# tx_buff   数据缓冲 | 必要参数 存放发送数据的缓冲区
# -----
spi2.write_readinto(tx_buff, rx_buff)
```

各版本的资源有所差异，请对照下表：

Board Type		144P-BTB				144P-2.54				100P-2.54			
HW-SPI	Logical	SCK	MOSI	MISO	CS0	SCK	MOSI	MISO	CS0	SCK	MOSI	MISO	CS0
LPSP11	id = 0					B10	B12	B13	B11	B10	B12	B13	B11
LPSP12	id = 1	C10	C12	C13	C11	C10	C12	C13	C11				
LPSP13	id = 2	B28	B30	B31	B29	B28	B30	B31	B29	B28	B30	B31	B29
LPSP14	id = 3	B18	B20	B21	B19	B18	B20	B21	B19	D0	D2	D3	D1

需要注意的是 **RT1021-144P-BTB** 核心板 **LPSP11** 不能使用。

需要注意的是 **RT1021-100P-2.54** 核心板 **LPSP12** 不能使用。

3.1.6. IIC 子模块

IIC 模块，基本兼容 MicroPython 的 IIC 模块：

```
# -----
# 构造接口 标准 MicroPython 的 machine.IIC 模块
# I2C_obj = I2C(id, freq = 400000)
# id          接口编号 | 必要参数 本固件支持 (0, 1, 3) 总共 3 个 I2C 模块
# freq        传输速率 | 可选参数 默认 400000
# return      返回内容 | 正常情况下返回对应 I2C 的对象
# -----
# 接口编号与对应 ID 和 引脚 对应表
from machine import I2C
iic4 = I2C(3, freq = 100000)
# -----
# 扫描 IIC 总线上是否有设备
# device_list = I2C.scan()
# return      返回内容 | 返回从 0x08 到 0x77 地址有响应的从机地址列表 字节数组形式
# -----
device_list = iic4.scan()
# -----
# 从指定地址的从机读取指定长度的数据
# rx_buff = I2C.readfrom(addr, lenght, stop = True)
# addr      目标地址 | 必要参数 目标通信器件的地址
# length     读取长度 | 必要参数 准备读取的数据长度
# stop       停止条件 | 可选参数 True 发送停止信号 False 不发生停止信号
# return     返回内容 | 返回读取到的内容 为字节数组形式
# -----
rx_byte = iic4.readfrom(device_list[0], 1, True)
# -----
# 从指定地址的从机读取缓冲区长度的数据
# I2C.readfrom_into(addr, rx_buff, stop = True)
# addr      目标地址 | 必要参数 目标通信器件的地址
# rx_buff    数据缓冲 | 必要参数 存放读取数据的缓冲区
# stop       停止条件 | 可选参数 True 发送停止信号 False 不发生停止信号
# -----
iic4.readfrom_into(device_list[0], rx_buff, True)
# -----
# 向指定地址的从机输出数据缓冲区长度数据
# I2C.writeto(addr, tx_buff, stop = True)
```

```
#      addr      目标地址 | 必要参数 目标通信器件的地址
#      tx_buff    数据缓冲 | 必要参数 存放发送数据的缓冲区
#      stop       停止条件 | 可选参数 True 发送停止信号 False 不发生停止信号
# -----
iic4.writeto(device_list[0], tx_buff, True)
# -----
# 向指定地址的从机输出矩阵数据
# I2C.writevto(addr, vectors, stop = True)
#      addr      目标地址 | 必要参数 目标通信器件的地址
#      vectors    矩阵缓冲 | 必要参数 存放发送数据的矩阵缓冲区
#      stop       停止条件 | 可选参数 True 发送停止信号 False 不发生停止信号
# -----
iic4.writevto(device_list[0], vectors, True)
```

各版本的资源有所差异，请对照下表：

Board Type		144P-BTB		144P-2.54		100P-2.54	
HW-I2C	Logical	SCL	SDA	SCL	SDA	SCL	SDA
LPI2C1	id = 0	B30	B31	B30	B31	B30	B31
LPI2C2	id = 1	C19	C18	C19	C18	C19	C18
LPI2C3	id = 2			B8	B9	B8	B9
LPI2C4	id = 3	D22	D23	D22	D23	D22	D23

需要注意的是 RT1021-144P-BTB 核心板 LPI2C3 不能使用。

不推荐使用 I2C 接口，因为它非常容易报错，一旦操作有误、通信线路受干扰，它就会直接报错停止运行。由于 NXP 固件中仅实现了硬件的 IIC 部分，软件 IIC 暂未支持，如果使用基本 I2C 操作接口进行自拟定的操作，也无法避免通信错误，并且效率极低。

3.2.NXP 的 smartcar 库

3.2.1.ticker 子模块

由于标准 machine 中的 Timer 类是基于软定时器实现，无法保证回调的实时性，因此 NXP 编写了 ticker 子模块，用于提供一个周期中断以保证可以有一个实时性较强的模块。

```
# -----
# 构造接口 实例化 PIT ticker 模块
# ticker_obj = ticker(id)
#     id          接口编号    |    必要参数 本固件支持 [0-3] 最多四个
#     return      返回内容    |    正常情况下返回对应 ticker 的对象
# -----
from smartcar import ticker
pit1 = ticker(1)

# -----
# 关联回调函数
# ticker.callback(handler)
#     handler      回调函数    |    必要参数 带自身对象传递的回调函数
# -----
ticker_flag = False
def time_pit_handler (ticker_obj): # 定义一个回调函数 必须有一个参数用于传递实例自身
    global ticker_flag # 需要注意的是这里得使用 global 修饰全局属性 否则它会新建一个局部变量
    ticker_flag = True
pit1.callback(time_pit_handler)

# -----
# 以指定周期启动 ticker
# ticker.start(period_ms)
#     period_ms    周期时间    |    必要参数 周期 毫秒单位 建议不低于 5ms
# -----
pit1.start(100)

# -----
# 停止 ticker
# ticker.stop()
# -----
pit1.stop()

# -----
# 获取计数值
# count = ticker.ticks()
#     return      返回内容    |    返回当前 ticker 的计数数值
# -----
count = pit1.ticks()
```

Python 本质上是解释型语言，无法保证强实时性无法保证数据采样的周期，这可能会导致例如：编码器采样周期短，而 Python 代码无法保证强实时性，导致编码器数据上下浮动。因此 NXP 提供了 caputer_list 用于进行底层自动采集，可以保证数据采集保持稳定的周期。

```
# -----
# 绑定捕获对象 最少一个最多八个 绑定后每个回调前都会先调用对象的 capture 方法接口
# ticker.capture_list(sensor_obj, ...)
#     sensor_obj 捕获对象 | 必要参数 Sensor 框架的传感器对象 最少一个 最多八个 (imu, ccd, key...)
#                               可关联 smartcar 的 ADC_Group_x 与 encoder_x
#                               可关联 seekfree 的 KEY_HANDLER, IMU660RX, IMU963RX, DL1X 和 TSL1401
# -----
pit1.capture_list(sensor_obj, ...)
```

需要注意的底层采集也是需要消耗时间的！例如，尽管 ADC_Group、encoder 模块的采集并不会过多消耗时间，但它也会占用一定的 ticker 模块的中断时间，而多个 TSL1401 的采集则会占用较长的采集时间。**需要自行考虑使用合适的 ticker 周期！**保险起见不要低于 5ms。

3.2.2.ADC_Group 子模块

通过 Python 代码调用 ADC 对象进行信号转换时需要通过解释器解释、对象查找、接口调用、底层转换、数据转换等步骤，这使得需要转换多个 ADC 数据时会浪费很多时间。

因此 NXP 提供了 ADC_Group 子模块，使得用户可以将同一个 ADC 模块下的引脚编入一个组中，通过组序列转换的方式一次性将多个通道数据完成转换，可以提高工作效率。

```
# -----
# 构造接口 用于构建一个 ADC_Group 对象
# ADC_Group_obj = ADC_Group(id)
#     id      模块索引 | 必要参数 RT1021 对应有 [1,2] 总共两个模块可选
#     return   返回内容 | 正常情况下返回对应 ADC_Group 的对象
# -----
from smartcar import ADC_Group
adc_group = ADC_Group(1)
# -----
# ADC_Group 添加对应引脚 需要该引脚是对应 ADC 模块的通道引脚
# ADC_Group.addch(pin_name)
#     pin_name 引脚名称 | 必要参数 引脚名称 本固件以核心板上引脚编号为准
# -----
adc_group.addch('B12')
# -----
# 直接调用的初始化接口 用于初始化制定的 ADC 硬件 需要注意本方法只能直接 ADC_Group.init 调用
# ADC_Group.init(id, period = ADC_Group.PMODE3, average = ADC_Group.AVG16)
#     id      模块索引 | 必要参数 RT1021 对应有 [1, 2] 总共两个模块可选
#     period   采样周期 | 可选参数 关键字输入 默认 ADC_Group.x, x = {PMODE0, PMODE1, PMODE2, PMODE3}
#     average  均值选项 | 可选参数 关键字输入 默认 ADC_Group.x, x = {AVG1, AVG4, AVG8, AVG16, AVG32}
# -----
ADC_Group.init(1, period = ADC_Group.PMODE3, average = ADC_Group.AVG16)
# -----
# 触发一次 ADC_Group 的转换 结果更新到数据缓冲区
# ADC_Group.capture()
```



```
# -----
adc_group.capture()
# -----
# 从 ADC_Group 数据缓冲区获取最新的数据
# data_buffer = ADC_Group.get()
# return 返回内容 | 返回当前 ADC_Group 的转换数值 为一个列表
# -----
data_buffer = adc_group.get()
# -----
# 触发一次 capture 方法接口 并从 ADC_Group 数据缓冲区获取最新的数据
# data_buffer = ADC_Group.read()
# return 返回内容 | 返回当前 ADC_Group 的转换数值 为一个列表
# -----
data_buffer = adc_group.read()
```

不过需要注意的是，**对应的 ADC_Group 子模块只能添加同个 ADC 模块下的通道**，也就是说，ADC_Group1 子模块只能添加 ADC1 的通道，ADC_Group2 子模块只能添加 ADC2 通道，两个组并不能相互混用。

3.2.3.encoder 子模块

由于标准 machine 中并没有 enc 类，所以无法进行编码器采集。因此 NXP 提供了 encoder 子模块用于进行编码器采集，支持正交编码器和带方向的增量式编码器。

```
# -----
# 构造接口 用于构建一个 encoder 对象
# encoder_obj = encoder(PhaseA, PhaseB, invert = False, capture_div = 1)
# PhaseA 引脚名称 | 必要参数 引脚名称字符串 编码器 A 相或 PLUS 引脚
# PhaseB 引脚名称 | 必要参数 引脚名称字符串 编码器 B 相或 DIR 引脚
# invert 模块索引 | 可选参数 是否反向 可以通过这个参数调整编码器旋转方向数据极性
# capture_div 模块索引 | 可选参数 关键字输入 设置采集触发分频
# return 返回内容 | 正常情况下返回对应 encoder 的对象
# -----
from smartcar import encoder
encoder_1 = encoder("D15", "D16", True, capture_div = 10)
# -----
# 增加一个 encoder 的采集请求 当达到 capture_div 数量时进行一次脉冲数据采集更新
# encoder.capture()
# -----
encoder_1.capture()
# -----
# 从 encoder 数据缓冲区获取最新的脉冲数据
# count = encoder.get()
# return 返回内容 | 返回当前 encoder 的计数数值
# -----
enc1_data = encoder_1.get()
# -----
# 增加一个 encoder 的采集请求 并从 encoder 数据缓冲区获取最新的脉冲数据
# count = encoder.read()
```

```
#      return      返回内容      |      返回当前 encoder 的计数数值
# -----
enc1_data = encoder_1.read()
# -----
# 通常使用时使用 Ticker 来进行自动采集
# 使用 ticker.capture_list(sensor_obj, ...) 来绑定自动采集对象
# -----
from smartcar import ticker
pit1 = ticker(1)
pit1.capture_list(encoder_1)
ticker_flag = False
def time_pit_handler (ticker_obj): # 定义一个回调函数 必须有一个参数用于传递实例自身
    global ticker_flag # 需要注意的是这里得使用 global 修饰全局属性 否则它会新建一个局部变量
    ticker_flag = True
pit1.callback(time_pit_handler)
pit1.start(10)
```

3.3. NXP 的 display 库

由于实际车模调试需要脱机操作, 那么添加一个屏幕用于显示调试参数变成了一个迫切的需求, 因此 NXP 提供了一个 display 模块兼容逐飞科技的 IPS200-SPI 串口屏幕用于显示。

```
# -----
#      构造接口 用于构建一个 LCD_Drv 对象
#      LCD_Drv(SPI_INDEX, BAUDRATE, DC_PIN, RST_PIN, LCD_TYPE)
#      SPI_INDEX  接口索引      |      必要参数 关键字输入 选择屏幕所用的 SPI 接口索引
#      BAUDRATE   通信速率      |      必要参数 关键字输入 SPI 的通信速率 最高 60MHz
#      DC_PIN     命令引脚      |      必要参数 关键字输入 一个 Pin 实例
#      RST_PIN    复位引脚      |      必要参数 关键字输入 一个 Pin 实例
#      LCD_TYPE   屏幕类型      |      必要参数 关键字输入 目前仅支持 LCD_Drv.LCD200_TYPE
#      return     返回内容      |      正常情况下返回对应 LCD_Drv 的对象
# -----
from machine import *
from display import *
cs = Pin('B29', Pin.OUT, value=True) # 定义片选引脚 拉高拉低一次 CS 片选确保屏幕通信时序正常
cs.high()
cs.low()
rst = Pin('B31', Pin.OUT, value=True) # 定义控制引脚 RET 复位控制
dc = Pin('B5', Pin.OUT, value=True) # 定义控制引脚 D C 命令控制
blk = Pin('C21', Pin.OUT, value=True) # 定义控制引脚 BLK 背光控制
drv = LCD_Drv(SPI_INDEX = 2, BAUDRATE = 6000000, DC_PIN = dc, RST_PIN = rst, LCD_TYPE = LCD_Drv.LCD200_TYPE)
# -----
#      构造接口 用于构建一个 LCD 对象
#      LCD(LCD_Drv)
#      LCD_Drv  接口对象      |      必要参数 LCD_Drv 对象
#      return   返回内容      |      正常情况下返回对应 LCD 的对象
# -----
lcd = LCD(drv)
# -----
#      修改 LCD 的前景色与背景色
#      LCD.color(pcolor, bgcolor)
#      pcolor   前景色      |      必要参数 RGB565 格式
#      bgcolor   背景色      |      必要参数 RGB565 格式
# -----
```

```

lcd.color(0xFFFF, 0x0000)
# -----
# 修改 LCD 的显示方向
# LCD.mode(dir)
#     dir          显示方向    |    必要参数 [0-竖屏, 1-横屏, 2-竖屏 180 旋转, 3-横屏 180 旋转]
# -----
lcd.mode(2)
# -----
# 清屏 不传入参数就使用当前的 背景色 清屏
# LCD.clear(color)
# LCD.clear()
#     color          颜色数值    |    非必要参数 RGB565 格式 输入参数则更新背景色并清屏
# -----
lcd.clear()
lcd.clear(0x0000)
# -----
# 各字体大小显示字符串
# LCD.str12(x, y, str, color = pcolor)
# LCD.str16(x, y, str, color = pcolor)
# LCD.str24(x, y, str, color = pcolor)
# LCD.str32(x, y, str, color = pcolor)
#     x              横轴坐标    |    必要参数 起始显示 X 坐标
#     y              纵轴坐标    |    必要参数 起始显示 Y 坐标
#     str            字符对象    |    必要参数 字符串
#     color          颜色数值    |    非必要参数 字符颜色 可以不填使用默认的前景色
# -----
lcd.str12(0, 0, "15={:b},{:d},{:o},{:x}".format(15,15,15,15), 0xF800)
lcd.str16(0, 12, "1.234={:}>.2f)".format(1.234), 0x07E0)
lcd.str24(0, 28, "123={:}<6d)".format(123), 0x001F)
lcd.str32(0, 52, "123={:}>6d)".format(123), 0xFFFF)
# -----
# 显示一条线
# LCD.line(x_star, y_star, x_end, y_end, color = pcolor, thick = 1)
#     x_star        横轴坐标    |    必要参数 起始 X 坐标
#     y_star        纵轴坐标    |    必要参数 起始 Y 坐标
#     x_end         横轴坐标    |    必要参数 结束 X 坐标
#     y_end         纵轴坐标    |    必要参数 结束 Y 坐标
#     color         颜色数值    |    非必要参数 线条颜色 可以不填使用默认的前景色
#     thick         线条粗细    |    非必要参数 线条粗细 默认一个像素宽度
# -----
lcd.line( 0, 84, 200, 16 + 84, color = 0xFFFF, thick = 1)
# -----
# 显示一个波形
# LCD.wave(x, y, width, high, data, max)
#     x              横轴坐标    |    必要参数 起始显示 X 坐标
#     y              纵轴坐标    |    必要参数 起始显示 Y 坐标
#     width          显示宽度    |    必要参数 波形显示窗口宽度 最好等同于数据个数
#     high          显示高度    |    必要参数 波形显示窗口高度
#     data           数据对象    |    必要参数 线条颜色 使用 unsigned short 类型的数组对象 兼容适配 TSL1401 数据列表
#     max           最大数值    |    必要参数 数据的最大值 用于进行波形高度的缩放
# -----
from array import *
arr = array('h', [0] * 128)
for i in range(0, 64):
    arr[i] = i * 64
    arr[127 - i] = i * 64
lcd.wave(0, 120, 128, 64, arr, max = 4095)

```

特别提示：不论你需要显示什么数据，对于 Python 来说，它们都是字符串！全部都使用 `LCD.str**(x, y, str, color)` 接口进行显示！

跟 `print` 函数一样，通过字符串格式化将数据整合为一个字符串对象即可！

跟 `print` 函数一样，通过字符串格式化将数据整合为一个字符串对象即可！

跟 `print` 函数一样，通过字符串格式化将数据整合为一个字符串对象即可！

3.4. 逐飞科技的 seekfree 库

由于 Python 是解释型语言，使用它来进行底层接口通信实现传感器驱动会非常的浪费性能，并且耗时会很长。因此基于 NXP 的 smartcar 库下 sensor 框架，逐飞科技提供了几个模块方便使用与调试（所用到的引脚暂时不可随意修改，使用固定的引脚）。

3.4.1. MOTOR_CONTROLLER 子模块

用于电机驱动，因为原生的 PWM 模块只能将 RT1021 的同个 PWM 子模块下的 A/B 相初始化为互补通道，而无法独立控制。但电机驱动通常需要两个独立控制的 PWM 通道，或者一个 PWM 通道一个 IO 信号。所以我们提供了这个电机驱动控制子模块，便于电机驱动控制。

可以用于直接驱动 DRV8701 单/双驱或者 HIP4082 单/双驱。

```
# -----
#  构造接口 用于构建一个 MOTOR_CONTROLLER 对象
#  MOTOR_CONTROLLER_obj = MOTOR_CONTROLLER(index, freq, duty = 0, invert = False)
#      index 电机索引 | 必要参数 通过 MOTOR_CONTROLLER.help() 查看
#      freq   信号频率 | 必要参数 PWM 信号的频率 范围是 [1 - 100000]
#      duty   占空比值 | 可选参数 关键字参数 默认为 0 范围 ±10000 正数正转 负数反转 正转反转方向取决于 invert
#      invert 反向设置 | 可选参数 关键字参数 是否反向 默认为 False 可以通过这个参数调整电机方向极性
#      return 返回内容 | 正常情况下返回对应 MOTOR_CONTROLLER 的对象
# -----
from seekfree import MOTOR_CONTROLLER
motor_1 = MOTOR_CONTROLLER(MOTOR_CONTROLLER.PWM_C3Q_DIR_C3I, 13000, duty = 0, invert = False)
# -----
#  更新或获取占空比值
#  MOTOR_CONTROLLER.duty(duty)
#  duty = MOTOR_CONTROLLER.duty()
#      duty      占空比值 | 可选参数 填数值就设置新的占空比 否则返回当前占空比 范围是 ±10000
#      return     返回内容 | 当不输入 duty 参数时 返回当前的占空比值 否则无返回值
# -----
motor_1.duty(-2500)
# -----
#  可以直接通过类调用 也可以通过对象调用 输出模块的使用帮助信息
#  MOTOR_CONTROLLER.help()
# -----
MOTOR_CONTROLLER.help()
motor_1.help()
# -----
#  通过对象调用 输出当前对象的自身信息
#  MOTOR_CONTROLLER.info()
# -----
motor_1.info()
```

3.4.1.1.RT1021-144P 芯片 BTB 连接器接口核心板固件支持引脚

支持学习板上的电机驱动信号接口, 引脚固定 ([C28/C29/C30/C31]、[D4/D5/D6/D7])。

驱动方式	PWM+DIR 组合驱动 (DRV8701)	PWM*2 组合驱动 (HIP4082)
引脚组合	MOTOR_CONTROLLER.PWM_C30_DIR_C31	MOTOR_CONTROLLER.PWM_C30_PWM_C31
	MOTOR_CONTROLLER.PWM_C28_DIR_C29	MOTOR_CONTROLLER.PWM_C28_PWM_C29
	MOTOR_CONTROLLER.PWM_D4_DIR_D5	MOTOR_CONTROLLER.PWM_D4_PWM_D5
	MOTOR_CONTROLLER.PWM_D6_DIR_D7	MOTOR_CONTROLLER.PWM_D6_PWM_D7

3.4.1.2.RT1021-144P 芯片 2.54 排针接口核心板固件支持引脚

支持学习板上的电机驱动信号接口, 引脚固定 ([C28/C29/C30/C31]、[D4/D5/D6/D7])。

驱动方式	PWM+DIR 组合驱动 (DRV8701)	PWM*2 组合驱动 (HIP4082)
引脚组合	MOTOR_CONTROLLER.PWM_C30_DIR_C31	MOTOR_CONTROLLER.PWM_C30_PWM_C31
	MOTOR_CONTROLLER.PWM_C28_DIR_C29	MOTOR_CONTROLLER.PWM_C28_PWM_C29
	MOTOR_CONTROLLER.PWM_D4_DIR_D5	MOTOR_CONTROLLER.PWM_D4_PWM_D5
	MOTOR_CONTROLLER.PWM_D6_DIR_D7	MOTOR_CONTROLLER.PWM_D6_PWM_D7

3.4.1.3.RT1021-100P 芯片 2.54 排针接口核心板固件支持引脚

支持学习板上的电机驱动信号接口, 引脚固定 ([C24/C25/C26/C27])。

驱动方式	PWM+DIR 组合驱动 (DRV8701)	PWM*2 组合驱动 (HIP4082)
引脚组合	MOTOR_CONTROLLER.PWM_C24_DIR_C26	MOTOR_CONTROLLER.PWM_C24_PWM_C26
	MOTOR_CONTROLLER.PWM_C25_DIR_C27	MOTOR_CONTROLLER.PWM_C25_PWM_C27

3.4.2.BLDC_CONTROLLER 子模块

用于直接驱动负压风扇无刷电调。

```
# -----
# 构造接口 用于构建一个 BLDC_CONTROLLER 对象
# BLDC_CONTROLLER_obj = BLDC_CONTROLLER(index, freq = 50, highlevel_us = 1000)
#     index      接口索引    | 必要参数 可选参数为 [PWM_C25, PWM_C27]
#     freq       信号频率    | 可选参数 关键字参数 PWM 频率 范围 50-300 默认 50
#     highlevel_us 高电平值   | 可选参数 关键字参数 初始的高电平时长 范围 [1000-2000] 默认 1000
#     return     返回内容    | 正常情况下返回对应 BLDC_CONTROLLER 的对象
# -----
from seekfree import BLDC_CONTROLLER
bldc1 = BLDC_CONTROLLER(BLDC_CONTROLLER.PWM_C25, freq = 300, highlevel_us = 1000)
# -----
# 更新或获取高电平时间值
# BLDC_CONTROLLER.highlevel_us(highlevel_us)
# highlevel_us = BLDC_CONTROLLER.highlevel_us()
#     highlevel_us 高电平值   | 可选参数 填数值就设置新的高电平时长 否则返回当前高电平时长 范围是 [1000-2000]
#     return       返回内容    | 当不输入 highlevel_us 参数时 返回当前的高电平时间值 否则无返回值
# -----
bldc1.highlevel_us(1750)
# -----
# 可以直接通过类调用 也可以通过对象调用 输出模块的使用帮助信息
# BLDC_CONTROLLER.help()
# -----
BLDC_CONTROLLER.help()
bldc1.help()
# -----
# 通过对象调用 输出当前对象的自身信息
# BLDC_CONTROLLER.info()
# -----
bldc1.info()
```

3.4.2.1.RT1021-144P 芯片 BTB 连接器接口核心板固件支持引脚

需要注意的是，C24 与 C25 属于同一个 PWM 子模块，C26 与 C27 属于同一个 PWM 子模块，因此使用时他们不能冲突，建议使用无刷电调时不使用 C25/C27 的 PWM 功能。

通道	引脚
1	BLDC_CONTROLLER.PWM_C25
2	BLDC_CONTROLLER.PWM_C27

3.4.2.2.RT1021-144P 芯片 2.54 排针接口核心板固件支持引脚

需要注意的是，C24 与 C25 属于同一个 PWM 子模块，C26 与 C27 属于同一个 PWM 子模块，因此使用时他们不能冲突，建议使用无刷电调时不使用 C25/C27 的 PWM 功能。

通道	引脚
1	BLDC_CONTROLLER.PWM_C25
2	BLDC_CONTROLLER.PWM_C27

3.4.2.3.RT1021-100P 芯片 2.54 排针接口核心板固件支持引脚

通道	引脚
1	BLDC_CONTROLLER.PWM_B26
2	BLDC_CONTROLLER.PWM_B27

3.4.3.KEY_HANDLER 子模块

四个按键的驱动，带消抖、长短按检测。默认为短按松发（按下后松开触发），默认消抖时长为 100ms，长按检测 500ms 生效。

```
# -----
# 构造接口 用于构建一个 KEY_HANDLER 对象
# KEY_HANDLER_obj = KEY_HANDLER(period)
#     period      扫描周期    | 必要参数 按键的扫描周期 毫秒单位 一般配合填写 Tickter 的运行周期
#     return      返回内容    | 正常情况下返回对应 KEY_HANDLER 的对象
# -----
from seekfree import KEY_HANDLER
key = KEY_HANDLER(10)
# -----
# 执行一次按键状态扫描
# KEY_HANDLER.capture()
# -----
key.capture()
# -----
# 获取当前四个按键状态 只需要一次 KEY_HANDLER.get() 后就不需要再调用这个接口
# key_state = KEY_HANDLER.get()
#     return      返回内容    | 返回当前四个按键状态 返回为一个列表
# -----
key_data = key.get()
```



```
# -----
# 触发一次 capture 方法接口 并获取当前四个按键状态
# key_state = KEY_HANDLER.read()
# return 返回内容 | 返回当前四个按键状态 返回为一个列表
# -----
key_data = key.read()
# -----
# 清除按键状态 长按会锁定长按状态不被清除
# KEY_HANDLER.clear(index)
# KEY_HANDLER.clear()
# index 按键序号 | 可选参数 [1, 4] 清除对应按键的触发状态 不输入参数则清空所有按键状态
# -----
key.clear()
key.clear(1)
# -----
# 获取当前按键实例的周期设置
# period = KEY_HANDLER.get_period()
# return 返回内容 | 返回当前按键实例的周期 毫秒单位
# -----
period = key.get_period()
# -----
# 可以直接通过类调用 也可以通过对象调用 输出模块的使用帮助信息
# KEY_HANDLER.help()
# -----
KEY_HANDLER.help()
key.help()
# -----
# 通过对象调用 输出当前对象的自身信息
# KEY_HANDLER.info()
# -----
key.info()
# -----
# 通常使用时使用 Ticker 来进行自动采集
# 使用 ticker.capture_list(sensor_obj, ...) 来绑定自动采集对象
# -----
from smartcar import ticker
pit1 = ticker(1)
pit1.capture_list(key)
ticker_flag = False
def time_pit_handler (ticker_obj): # 定义一个回调函数 必须有一个参数用于传递实例自身
    global ticker_flag # 需要注意的是这里得使用 global 修饰全局属性 否则它会新建一个局部变量
    ticker_flag = True
pit1.callback(time_pit_handler)
pit1.start(10)
```

默认按键连接为释放高电平，按下低电平。各版本核心板固件支持引脚：

核心板	KEY1	KEY2	KEY3	KEY4
RT1021-144P 芯片 BTB 连接器接口核心板	C8	C9	C14	C15
RT1021-144P 芯片 2.54 排针接口核心板	C8	C9	C14	C15

RT1021-100P 芯片 2.54 排 针接口核心板	D20	D21	D22	D23
---------------------------------	-----	-----	-----	-----

3.4.4.IMU660/963RX 子模块

用于驱动 IMU[660/963]RX (X=[A, B, ...]) 系列的六轴或九轴姿态传感器模块，使用固定引脚（SCK-C10 / MOSI-C12 / MISO-C13 / CS-C11），推荐挂载在 Ticker 下自动采集。

```
# -----
# 构造接口 用于构建一个 IMU660RX / IMU963RX 对象
# imu_obj = IMU[660/963]RX (capture_div = 1,
#                             imu_type = IMU[660/963]RX.TYPE_AUTO,
#                             quar_rate = IMU[660/963]RX.RATE_DISABLE)
# imu_obj = IMU[660/963]RX()
# capture_div 采集分频 | 可选关键字参数 默认为 1 也就是每次都采集 代表多少次触发进行一次采集
# imu_type 模块类型 | 可选关键字参数 IMU[660/963]RX.TYPE_x, x = (AUTO, RA, RB, RC) 默认 AUTO
# quar_rate 硬解频率 | 可选关键字参数 IMU[660/963]RX.RATE_x, x = (15HZ, 30HZ, 60HZ, 120HZ,
#                             240HZ, 480HZ, DISABLE) 默认 DISABLE
# return 返回内容 | 正常情况下返回对应 IMU[660/963]RX 的对象
# -----
from seekfree import IMU[660/963]RX
imu = IMU660RX() / imu = IMU963RX()
# -----
# 增加一个 IMU 的采集请求 当达到 capture_div 数量时进行一次采集并将数据缓存
# IMU[660/963]RX.capture()
# -----
imu.capture()
# -----
# 获取加速度计和陀螺仪数据缓冲区
# data_list = IMU[660/963]RX.get()
# return 返回内容 | 返回加速度计和陀螺仪数据缓冲区为一个列表
#                             顺序{acc_x, acc_y, acc_z, gyro_x, gyro_y, gyro_z}
# -----
data_list = imu.get()
# -----
# 获取欧拉角数据缓冲区
# data_list = IMU[660/963]RX.get_euler()
# return 返回内容 | 返回欧拉角数据缓冲区为一个列表 顺序 {roll, pitch, yaw}
# -----
data_list = imu.get_euler()
# -----
# 获取四元数数据缓冲区
# data_list = IMU[660/963]RX.get_quaternion()
# return 返回内容 | 返回四元数数据缓冲区为一个列表 顺序 {x, y, z, w}
# -----
data_list = imu.get_quaternion()
# -----
# 立即进行一次 capture 并从 IMU 数据缓冲区获取最新的数据
# data_list = IMU[660/963]RX.read()
# return 返回内容 | 返回当前 IMU 六轴数据 返回为一个列表
# -----
data_list = imu.read()
```

```
# -----
# 可以直接通过类调用 也可以通过对象调用 输出模块的使用帮助信息
# IMU[660/963]RX.help()
# -----
IMU660RX.help() / IMU963RX.help()
imu.help()
# -----
# 通过对象调用 输出当前对象的自身信息
# IMU[660/963]RX.info()
# -----
imu.info()
# -----
# 通常使用时使用 Ticker 来进行自动采集
# 使用 ticker.capture_list(sensor_obj, ...) 来绑定自动采集对象
# -----
from smartcar import ticker
pit1 = ticker(1)
pit1.capture_list(imu)
ticker_flag = False
def time_pit_handler (ticker_obj): # 定义一个回调函数 必须有一个参数用于传递实例自身
    global ticker_flag # 需要注意的是这里得使用 global 修饰全局属性 否则它会新建一个局部变量
    ticker_flag = True
pit1.callback(time_pit_handler)
pit1.start(10)
```

可以通过 IMUxxxRX.help()查看当前版本的固件到底支持哪些模块型号。各版本核心板固

件支持引脚：

核心板	SPI	SPC	SDI	SDO	CS	INT2
RT1021-144P 芯片 BTB 连接器接口核心板	LPSPi2	C10	C12	C13	C11	D19
RT1021-144P 芯片 2.54 排针接口核心板	LPSPi2	C10	C12	C13	C11	D19
RT1021-100P 芯片 2.54 排针接口核心板	LPSPi1	B10	B12	B13	B11	C21

需要注意的是，IMU660RX/ IMU963RX 使能硬件解算时，必须要连接 INT2 引脚。随后固件会自动通过 INT2 硬件触发外部中断进行全自动解算，不再需要 Ticker 来触发软件采集，数据更新频率参照设置的硬解频率。

也就是说，如果要使用硬件解算，那么就连接 INT2 引脚，并删除 Ticker.capture_list()关联，让其自己通过 INT2 进行硬件自动触发采集。

3.4.5.DL1X 子模块

用于驱动 DL1X (X=[A, B, ...]) 系列 ToF 红外激光测距模块, 使用固定引脚 (SCL-D0 / SDA-D1 / XS-D2), 推荐挂载在 Ticker 下自动采集, 但需要注意采集频率不能高于模块频率。

```
# -----  
# 构造接口 用于构建一个 DL1X 对象  
# DL1X_obj = DL1X(capture_div = 1)  
# DL1X_obj = DL1X()  
# capture_div 采集分频 | 非必要参数 默认为 1 也就是每次都采集 代表多少次触发进行一次采集  
# return 返回内容 | 正常情况下返回对应 DL1X 的对象  
# -----  
from seekfree import DL1X  
tof = DL1X()  
# -----  
# 增加一个 DL1X 的采集请求 当达到 capture_div 数量时进行一次采集并将数据缓存  
# DL1X.capture()  
# -----  
tof.capture()  
# -----  
# 从 DL1X 数据缓冲区获取最新的数据  
# distance_mm = DL1X.get()  
# return 返回内容 | 返回当前 DL1X 距离数据 单位 mm  
# -----  
distance_mm = tof.get()  
# -----  
# 立即进行一次 capture 并从 IMU 数据缓冲区获取最新的数据  
# distance_mm = DL1X.read()  
# return 返回内容 | 返回当前 DL1X 距离数据 单位 mm  
# -----  
distance_mm = tof.read()  
# -----  
# 可以直接通过类调用 也可以通过对象调用 输出模块的使用帮助信息  
# DL1X.help()  
# -----  
DL1X.help()  
tof.help()  
# -----  
# 通过对象调用 输出当前对象的自身信息  
# DL1X.info()  
# -----  
tof.info()  
# -----  
# 通常使用时使用 Ticker 来进行自动采集  
# 使用 ticker.capture_list(sensor_obj, ...) 来绑定自动采集对象  
# -----  
from smartcar import ticker  
pit1 = ticker(1)  
pit1.capture_list(tof)  
ticker_flag = False  
def time_pit_handler (ticker_obj): # 定义一个回调函数 必须有一个参数用于传递实例自身  
    global ticker_flag # 需要注意的是这里得使用 global 修饰全局属性 否则它会新建一个局部变量  
    ticker_flag = True
```

```
pit1.callback(time_pit_handler)
pit1.start(50)
```

各版本核心板固件支持引脚：

核心板	SCL	SDA	XS	INT
RT1021-144P 芯片 BTB 连接器接口核心板	D0	D1	D2	D3
RT1021-144P 芯片 2.54 排针接口核心板	D0	D1	D2	D3
RT1021-100P 芯片 2.54 排针接口核心板	C22	C23	B4	B5

3.4.6. TSL1401 子模块

用于驱动 TSL1401 红孩儿线阵 CCD 模块。推荐挂载在 Ticker 下自动采集，可以调整采集分频控制曝光时间，不过建议保持默认分频，选择合适的 ticker 周期作为曝光周期。

```
# -----
# 构造接口 用于构建一个 TSL1401 对象
# TSL1401_obj = TSL1401(capture_div)
#     capture_div    采集分频    | 非必要参数 默认为 1 也就是每次都采集 代表多少次触发进行一次采集
#     return         返回内容    | 正常情况下返回对应 TSL1401 的对象
# -----
from seekfree import TSL1401
ccd = TSL1401(10)
# -----
# 设置 TSL1401 的转换精度
# TSL1401.set_resolution(resolution)
#     resolution      精度参数    | 必要参数 TSL1401.x , x = {RES_8BIT, RES_10BIT, RES_12BIT}
# -----
ccd.set_resolution(resolution)
# -----
# 增加一个 TSL1401 的采集请求 当达到 capture_div 数量时进行一次采集并将数据缓存
# TSL1401.capture()
# -----
ccd.capture()
# -----
# 从 TSL1401 数据缓冲区获取最新的数据
# data_buffer = TSL1401.get()
#     return         返回内容    | 返回当前 TSL1401 的转换数值 为一个列表
# -----
data_buffer = ccd.get()
# -----
# 立即进行一次 capture 并从 TSL1401 数据缓冲区获取最新的数据
# data_buffer = TSL1401.read()
#     return         返回内容    | 返回当前 TSL1401 的转换数值 为一个列表
# -----
data_buffer = ccd.read()
```

```
# -----
# 可以直接通过类调用 也可以通过对象调用 输出模块的使用帮助信息
# TSL1401.help()
# -----
TSL1401.help()
ccd.help()
# -----
# 通过对象调用 输出当前对象的自身信息
# TSL1401.info()
# -----
ccd.info()
# -----
# 通常使用时使用 Ticker 来进行自动采集
# 使用 ticker.capture_list(sensor_obj, ...) 来绑定自动采集对象
# -----
from smartcar import ticker
pit1 = ticker(1)
pit1.capture_list(ccd)
ticker_flag = False
def time_pit_handler (ticker_obj): # 定义一个回调函数 必须有一个参数用于传递实例自身
    global ticker_flag # 需要注意的是这里得使用 global 修饰全局属性 否则它会新建一个局部变量
    ticker_flag = True
pit1.callback(time_pit_handler)
pit1.start(10)
```

各版本核心板固件支持引脚：

核心板	CLK	SI	A01	A02	A03	A04
RT1021-144P 芯片 BTB 连接器接口核心板	B13	B16	B22	B23	B24	B25
RT1021-144P 芯片 2.54 排针接口核心板	B13	B16	B22	B23	B24	B25
RT1021-100P 芯片 2.54 排针接口核心板	B3	C8	B29	B31	-	-

数据以元组方式获取，可以显示在屏幕上或通过无线模块发送到逐飞助手：

```
# 显示到 display IPS200-SPI 屏幕：
... # 代码略
ccd_data1 = ccd.get(0)
lcd.wave(0, 0, 128, 64, ccd_data1, max = 4095)
... # 代码略

# 显示到 seekfree IPS200PRO 屏幕：
... # 代码略
ccd_data1 = ccd.get(0)
data_buffer1 = array('h', [0] * (128))
for i in range (128):
    data_buffer1[i] = int(ccd_data1[i] / 40.95)
ips200pro.waveform_value(waveform_id, 1, data_buffer1, 0xF800)
... # 代码略

# 通过 seekfree WIRELESS_UART 无线串口模块 发送到逐飞助手：
```

```

...                               # 代码略
ccd_data1 = ccd.get(0)
wireless.send_ccd_image(WIRELESS_UART.CCD1_BUFFER_INDEX)
...                               # 代码略

# 通过 seekfree WIFI_SPI WiFi-SPI 模块 发送到逐飞助手:
...                               # 代码略
ccd_data1 = ccd.get(0)
wifi.send_ccd_image(WIFI_SPI.CCD3_4_BUFFER_INDEX)
...                               # 代码略

```

3.4.7. 传感器子模块与 ticker 的 caputer_list 关联采集说明

在前面《3.2.3.ticker 子模块》中提到过“NXP 提供了 caputer_list 用于进行底层自动采集”。

在实际应用时，需要考虑各模块的采集周期、模块采集耗时，还要考虑自己的回调代码需要消耗多少时间。这里给出一个参考：

首先使用 4 个 CCD 模块，一个 IMU 模块，一个 ToF 模块，并且使用 KEY_HANDLER 子模块，再加上两个轮子各自一个 encoder 模块，总共六个需要挂载自动采集的模块。

然后检查这几个模块中更新频率最低的，是 ToF 模块，如果使用 DL1B 则 100Hz 更新频率，10ms 的周期，如果使用 DL1A 模块则 30Hz 更新频率，要至少 33ms 的周期。这里就考虑使用 10ms 作为 ticker 的触发周期。使用 DL1B 时正好匹配频率周期，使用 DL1A 时进行 4 分频采集即可，等效 25Hz 频率 40ms 周期。

这样 TSL1401、IMU 可以直接不分频挂载，一来是 IMU 的输出频率高于 100Hz（陀螺仪数据输出频率 200+Hz），二来 CCD 的曝光时间差不多正好，也兼顾图像刷新帧率，亮度不够可以补光。

```

from machine import *
from smartcar import *
from seekfree import *

ccd = TSL1401()                # 实例化模块对象 采集不分频 采集周期为 10ms （ticker 周期）
imu = IMU963RA()              # 实例化模块对象 采集不分频 采集周期为 10ms （ticker 周期）
tof = DL1B()                  # 实例化模块对象 采集不分频 采集周期为 10ms （ticker 周期）

key = KEY_HANDLER(10)         # 这里填入 10ms （ticker 周期） 的周期值

```

```

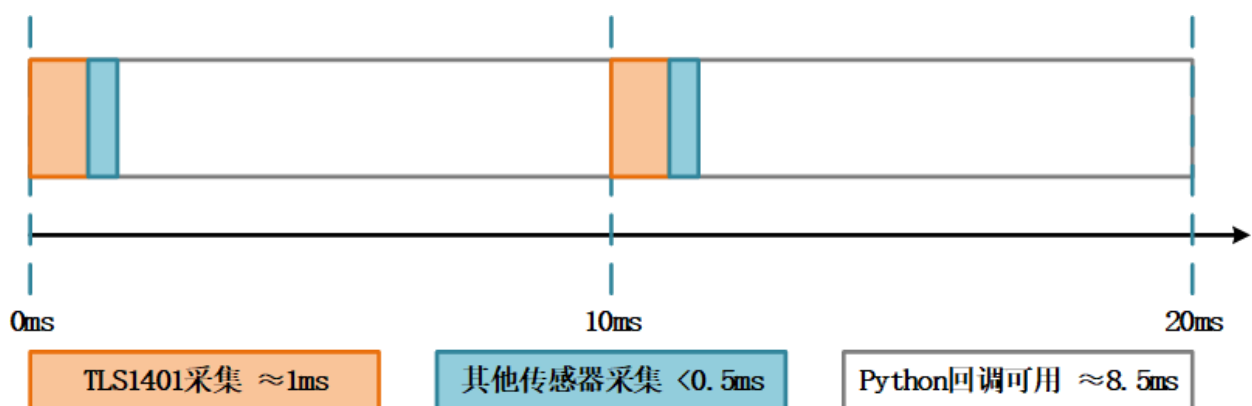
encoder_1 = encoder("D15", "D16", True) # 实例化编码器对象 反向
encoder_2 = encoder("D13", "D14")      # 实例化编码器对象

ticker_flag = False
def time_pit_handler (time):           # 定义一个回调函数 需要一个参数 这个参数就是 ticker 实例自身
    global ticker_flag                 # 需要注意的是这里得使用 global 修饰全局属性
    ticker_flag = True                 # 否则它会新建一个局部变量
    ...                                # 回调处理代码略

pit = ticker(1)                         # 实例化 ticker 模块
pit.callback(time_pit_handler)         # 绑定回调函数
# 将各模块挂载到 ticker 的 capture_list 中 绑定顺序就是采集顺序
pit.capture_list(ccd, imu, tof, key, encoder_1, encoder_2)
pit.start(10)                          # 以 10ms 周期启动 ticker 模块

```

那么此时 ticker 底层触发与采集状态大致情况为(此处时间仅为参考并非真实采样时间):



如果使用分频去调节曝光或者采集, 就可能会导致 Python 的回调函数可用时间不确定,

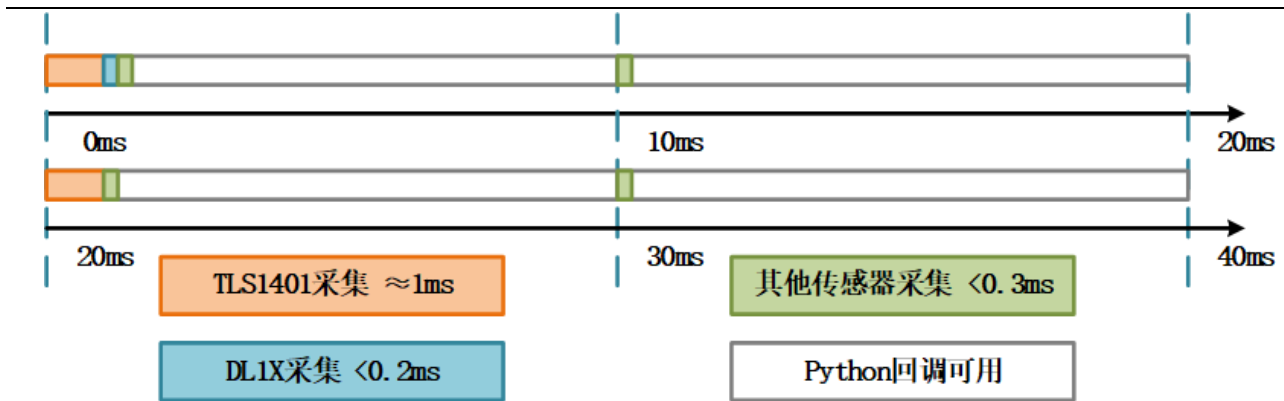
例如如下的代码设置, 修改了 TSL1401 和 DL1B 的采集分频:

```

... # 代码略
ccd = TSL1401(2) # 实例化模块对象 采集 2 分频 采集周期为 2 * 10ms (ticker 周期)
imu = IMU963RA() # 实例化模块对象 采集不分频 采集周期为 10ms (ticker 周期)
tof = DL1B(4) # 实例化模块对象 采集 4 分频 采集周期为 4 * 10ms (ticker 周期)
... # 代码略

```

那么此时 ticker 底层触发与采集状态大致情况为如下四个周期的循环(此处时间仅为参考并非真实采样时间):



这样就会导致回调可用时间不固定，不过也并不会过多的影响采集实时性和周期一致性，需要自行进行规划考虑。

3.4.8.WIRELESS_UART 子模块

用于驱动逐飞科技的无线转串口模块，**用于对接逐飞助手上位机 V2**，方便进行调试。

如果使用 V2.4 版本以上无线串口模块，可以任意设置波特率，模块会自动识别并匹配，如果使用 V2.4 以下版本，需要自行通过上位机设置波特率后程序设置对应的波特率才能通信。

```
# -----
# 构造接口 用于构建一个 WIRELESS_UART 对象
# WIRELESS_UART_obj = WIRELESS_UART(baudrate = 460800)
#     baudrate    传输速率    |    可选参数 默认 460800
#     return      返回内容    |    正常情况下返回对应 WIRELESS_UART 的对象
# -----
from seekfree import WIRELESS_UART
wireless = WIRELESS_UART(460800)
# -----
# 发送字符串
# WIRELESS_UART.send_str(str)
#     str          字符对象    |    必要参数 字符串
# -----
wireless.send_str("hello world.\r\n")
# -----
# 查询接收字节数组 返回值是实际接收的长度
# WIRELESS_UART.receive_bytearray (array, length)
#     array        字节数组    |    可以通过 import array 后新建一个 'b' 类型字节数组
#     length       读取长度    |    不能超过字节数组的最大长度
# -----
wireless.receive_bytearray (array, length)
# -----
# 发送字节数组
# WIRELESS_UART.send_bytearray (array, length)
#     array        字节数组    |    可以通过 import array 后新建一个 'b' 类型字节数组
#     length       发送长度    |    不能超过字节数组的最大长度
# -----
```

```
wireless.send_bytearray(array, length)
# -----
# 逐飞助手虚拟示波器数据上传
# WIRELESS_UART.send_oscilloscope(d1,[d2, d3, d4, d5, d6, d7, d8])
# dx          波形数据    |    至少一个数据 最多可以填八个数据 数据类型支持浮点数
# -----
wireless.send_oscilloscope(3.14, 16.125)
# -----
# 将对应编号的 CCD 数据上传到逐飞助手
# WIRELESS_UART.send_ccd_image(index)
# index      数据索引    |    可选参数共有六个 WIRELESS_UART.
#                                [CCD1_BUFFER_INDEX ,  CCD2_BUFFER_INDEX ]
#                                [CCD3_BUFFER_INDEX ,  CCD4_BUFFER_INDEX ]
#                                分别代表 (选择 CCD[1, 4] 单个)图像一起显示
# -----
wireless.send_ccd_image(CCD1_BUFFER_INDEX)
# -----
# 逐飞助手调参数据解析 会返回八个标志位的列表 标识各通道是否有数据更新
# data_flag = WIRELESS_UART.data_analysis()
# return      返回内容    |    返回当前 WIRELESS_UART 接收缓冲区八个标志位 为一个列表
# -----
data_flag = wireless.data_analysis()
# -----
# 逐飞助手调参数据获取 会返回八个数据的列表
# data_buffer = WIRELESS_UART.get_data()
# return      返回内容    |    返回当前 WIRELESS_UART 接收缓冲区八个数据 为一个列表
# -----
data_buffer = wireless.get_data()
# -----
# 可以直接通过类调用 也可以通过对象调用 输出模块的使用帮助信息
# WIRELESS_UART.help()
# -----
WIRELESS_UART.help()
wireless.help()
# -----
# 通过对象调用 输出当前对象的自身信息
# WIRELESS_UART.info()
# -----
wireless.info()
```

各版本核心板固件支持引脚：

核心板	UART	(模块的)TX	(模块的)RX	RTS
RT1021-144P 芯片 BTB 连接器接口核心板	LPUART3	C7	C6	C5
RT1021-144P 芯片 2.54 排针接口核心板	LPUART3	C7	C6	C5
RT1021-100P 芯片 2.54 排针接口核心板	LPUART3	C7	C6	D24

3.4.9.WIFI_SPI 子模块

用于驱动逐飞科技的 WIFI-SPI 模块，**用于对接逐飞助手上位机 V2**，方便进行调试。

```
# -----
# 构造接口 用于构建一个 WIFI_SPI 对象
# WIFI_SPI_obj = WIFI_SPI(wifi_ssid, pass_word, connect_type, ip_addr, connect_port)
#     wifi_ssid      热点名称    | 必要参数 WiFi 热点名称 字符串
#     pass_word      热点密码    | 必要参数 WiFi 热点密码 字符串
#     connect_type    连接类型    | 必要参数 连接类型 WIFI_SPI.TCP_CONNECT / WIFI_SPI.UDP_CONNECT
#     ip_addr         连接地址    | 必要参数 目标连接地址 字符串
#     connect_port    连接端口    | 必要参数 目标连接端口 字符串
#     return          返回内容    | 正常情况下返回对应 WIFI_SPI 的对象
# -----
from seekfree import WIFI_SPI
wifi = WIFI_SPI("WIFI_NAME", "WIFI_PASSWORD", WIFI_SPI.TCP_CONNECT, "192.168.1.13", "8086")
# -----
# 发送字符串
# WIFI_SPI.send_str(str)
#     str            字符对象    | 必要参数 字符串
# -----
wifi.send_str("hello world.\r\n")
# -----
# 查询接收字节数组 返回值是实际接收的长度
# WIFI_SPI.receive_bytearray (array, length)
#     array          字节数组    | 可以通过 import array 后新建一个 'b' 类型字节数组
#     length         读取长度    | 不能超过字节数组的最大长度
# -----
wifi.receive_bytearray (array, length)
# -----
# 发送字节数组
# WIFI_SPI.send_bytearray (array, length)
#     array          字节数组    | 可以通过 import array 后新建一个 'b' 类型字节数组
#     length         发送长度    | 不能超过字节数组的最大长度
# -----
wifi.send_bytearray (array, length)
# -----
# 逐飞助手虚拟示波器数据上传
# WIFI_SPI.send_oscilloscope(d1,[d2, d3, d4, d5, d6, d7, d8])
#     dx            波形数据    | 至少一个数据 最多可以填八个数据 数据类型支持浮点数
# -----
wifi.send_oscilloscope(3.14, 16.125)
# -----
# 将对应编号的 CCD 数据上传到逐飞助手
# WIFI_SPI.send_ccd_image(index)
#     index         数据索引    | 可选参数共有六个 WIFI_SPI.
#                                     [CCD1_BUFFER_INDEX , CCD2_BUFFER_INDEX ]
#                                     [CCD3_BUFFER_INDEX , CCD4_BUFFER_INDEX ]
#                                     分别代表 (选择 CCD[1, 4] 单个)图像一起显示
# -----
wifi.send_oscilloscope(CCD1_BUFFER_INDEX)
# -----
# 逐飞助手调参数据解析 会返回八个标志位的列表 标识各通道是否有数据更新
# data_flag = WIFI_SPI.data_analysis()
#     return        返回内容    | 返回当前 WIRELESS_UART 接收缓冲区八个标志位 为一个列表
```

```
# -----
data_flag = wifi.data_analysis()
# -----
# 逐飞助手调参数据获取 会返回八个数据的列表
# data_buffer = WIFI_SPI.get_data()
# return 返回内容 | 返回当前 WIRELESS_UART 接收缓冲区八个数据 为一个列表
# -----
data_buffer = wifi.get_data()
# -----
# 可以直接通过类调用 也可以通过对象调用 输出模块的使用帮助信息
# WIFI_SPI.help()
# -----
WIFI_SPI.help()
wifi.help()
# -----
# 通过对象调用 输出当前对象的自身信息
# WIFI_SPI.info()
# -----
wifi.info()
```

各版本核心板固件支持引脚：

核心板	SPI	SCK	MOSI	MISO	CS	INT	RST
RT1021-144P 芯片 BTB 连接器接口核心板	LPSPi4	B18	B20	B21	B19	B3	B4
RT1021-144P 芯片 2.54 排针接口核心板	LPSPi4	B18	B20	B21	B19	B3	B4
RT1021-100P 芯片 2.54 排针接口核心板	-	-	-	-	-	-	-

3.4.10.IPS200PRO 子模块

该子模块是为了支持最新的 IPS200PRO 串口中文彩屏,各版本核心板固件支持引脚：

核心板	SPI	CLK	MOSI	RST	INT	CS	BLK
RT1021-144P 芯片 BTB 连接器接口核心板	LPSPi3	B28	B30	B31	B5	B29	C21
RT1021-144P 芯片 2.54 排针接口核心板	LPSPi3	B28	B30	B31	B5	B29	C21
RT1021-100P 芯片 2.54 排针接口核心板	LPSPi3	B28	B30	B9	B8	C5	B5

由于 IPS200PRO 接口较多，因此会分成多个部分进行说明。

```
# -----
# 构造接口 用于构建一个 IPS200PRO 对象
```

```
# ips200pro_obj = IPS200PRO(title_position = IPS200PRO.TITLE_BOTTOM, title_high = 30)
#     title_position      标题位置      | 必要参数 IPS200PRO.TITLE_xxx
#                               参数 xxx = (LEFT, RIGHT, TOP, BOTTOM) 对应 (左侧, 右侧, 上方, 下方) 默认为下方
#     title_high          标题高度      | 必要参数 标题栏的高度值 数值范围在 [1, 200]
#     return              返回内容      | 正常情况下返回对应 IPS200PRO 的对象
# -----
from seekfree import IPS200PRO
ips200pro = IPS200PRO(IPS200PRO.TITLE_BOTTOM, 30)
# -----
# 可以直接通过类调用 也可以通过对象调用 输出模块的使用帮助信息
# IPS200PRO.help()
# -----
IPS200PRO.help()
ips200pro.help()
# -----
# 通过对象调用 输出当前对象的自身信息
# IPS200PRO.info()
# -----
ips200pro.info()
# -----
# RGB888 转换 RGB565 接口
# rgb565_16bit = IPS200PRO.rgb888_to_rgb565(rgb888_24bit)
#     rgb888_24bit        颜色数值      | 必要参数 一个 RGB888 格式的 24bit 色彩值
#     return              返回内容      | 返回一个 RGB565 格式的 16bit 色彩值
#     rgb565_16bit = IPS200PRO.rgb888_to_rgb565(red_8bit, green_8bit, blue_8bit)
#     red_8bit            红色数值      | 必要参数 一个 RGB888 格式的 8bit R 色彩值分量
#     green_8bit          绿色数值      | 必要参数 一个 RGB888 格式的 8bit G 色彩值分量
#     blue_8bit           蓝色数值      | 必要参数 一个 RGB888 格式的 8bit B 色彩值分量
#     return              返回内容      | 返回一个 RGB565 格式的 16bit 色彩值
# -----
rgb565_16bit = ips200pro.rgb888_to_rgb565(0x39C5BB)
rgb565_16bit = ips200pro.rgb888_to_rgb565(0x39, 0xC5, 0xBB)
# -----
# 修改组件字体 驱动默认的字大小是 16 可以在 ips200pro.info() 查看
# IPS200PRO.set_font(widgets_id, font)
#     widgets_id          组件索引      | 必要参数 组件的索引号 索引为 0 时修改默认字体大小
#     font                字体大小      | 必要参数 IPS200PRO.FONT_SIZE_xxx
#                               参数 xxx = (12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 32, 34, 36, 40)
#                               需要注意 仅 16、20、24 号字体支持中文显示
# -----
ips200pro.set_font(0, IPS200PRO.FONT_SIZE_24)
# -----
# 修改屏幕显示方向
# IPS200PRO.set_dir(dir)
#     dir                显示方向      | 必要参数 屏幕显示方向 IPS200PRO.xxx
#                               可用参数 xxx = (PORTRAIT, PORTRAIT_180, CROSSWISE, CROSSWISE_180)
#                               对应 (竖屏, 反转竖屏, 横屏, 反转横屏)
# -----
ips200pro.set_dir(IPS200PRO.CROSSWISE)
# -----
# 修改背光亮度
# IPS200PRO.set_backlight(backlight)
#     backlight          背光亮度      | 必要参数 背光亮度值 数值范围在 [1, 255]
# -----
ips200pro.set_backlight(50)
```

3.4.10.1.page 页面相关接口

页面是 IPS200PRO 屏幕的显示基础，所有显示组件都需要以页面为基础新建。

```
# -----
# 新建一个 页面 返回 页面 索引号
# page_id = IPS200PRO.page_create(page_name)
#     page_name  页面名称    |    必要参数 支持中文英文 UTF-8 格式
#     return      返回内容    |    正常情况下返回对应 页面 的索引
# -----
page_id1 = ips200pro.page_create("page1")
# -----
# 通过对象调用 输出模块的 page 部分的使用帮助信息
# IPS200PRO.help_page ()
# -----
ips200pro.help_page ()
# -----
# 修改 页面 名称
# IPS200PRO.page_name(page_id, page_name)
#     page_id      页面索引    |    必要参数 页面的索引号
#     page_name     页面名称    |    必要参数 支持中文英文 UTF-8 格式
# -----
ips200pro.page_name(page_id1, "页面 1")
# -----
# 切换到指定 页面
# IPS200PRO.page_switch(page_id, anim_enable = IPS200PRO.PAGE_ANIM_OFF)
#     page_id      页面索引    |    必要参数 页面的索引号
#     anim_enable   动画使能    |    可选参数 IPS200PRO.(PAGE_ANIM_OFF, PAGE_ANIM_ON) 默认 PAGE_ANIM_OFF 关闭动画
# -----
ips200pro.page_switch(page_id1)
# -----
# 修改组件字体 驱动默认的字大小是 16 可以在 ips200pro.info() 查看
# IPS200PRO.set_font(widgets_id, font)
#     widgets_id   组件索引    |    必要参数 组件的索引号 索引为 0 时修改默认字体大小
#     font          字体大小    |    必要参数 IPS200PRO.FONT_SIZE_xxx
#                                     参数 xxx = (12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 32, 34, 36, 40)
#                                     需要注意 仅 16、20、24 号字体支持中文显示
# -----
ips200pro.set_font(page_id1, IPS200PRO.FONT_SIZE_20) # 由于页面组件的特殊性 修改任意页面字体会对所有页面标题字体生效
# -----
# 修改组件颜色
# IPS200PRO.set_color(widgets_id, type, color)
#     widgets_id   组件索引    |    必要参数 组件 的索引号
#     type          设置选项    |    必要参数 指定要设置的是什么的颜色 IPS200PRO.COLOR_xxx
#                                     PAGE 可用参数 xxx = (FOREGROUND, BACKGROUND, PAGE_SELECTED_TEXT, PAGE_SELECTED_BG)
#                                     对应 (前景色, 背景色, 选中页面后的标题文字颜色, 选中页面后的标题背景颜色)
#     color         颜色数值    |    必要参数 RGB565 格式的颜色
# -----
# 由于页面组件的特殊性 修改任意页面标题颜色会对所有页面标题颜色生效
ips200pro.set_color(page_id1, IPS200PRO.COLOR_FOREGROUND, ips200pro.rgb888_to_rgb565(0x39C5BB))
ips200pro.set_color(page_id1, IPS200PRO.COLOR_BACKGROUND, ips200pro.rgb888_to_rgb565(0xA5A5A5))
ips200pro.set_color(page_id1, IPS200PRO.COLOR_PAGE_SELECTED_TEXT, 0xFFFF)
ips200pro.set_color(page_id1, IPS200PRO.COLOR_PAGE_SELECTED_BG , 0x0000)
# -----
# 设置组件隐藏显示
```

```
# IPS200PRO.set_hidden(widgets_id, enable)
#     widgets_id  组件索引    |    必要参数  组件  的索引号
#     enable      隐藏使能    |    必要参数  True  隐藏  False  取消隐藏
# -----
ips200pro.set_hidden(page_id1, True)
```

3.4.10.2.label 标签相关接口

标签是基础文本显示组件，可以用来显示文本信息，也可以设置为其它组件的子对象来组合使用，例如设置为表格的子对象，实现单个单元格分隔显示两组不同的信息，或者设置为进度条的子对象，用来显示进度百分比等。

```
from seekfree import IPS200PRO
ips200pro = IPS200PRO(IPS200PRO.TITLE_BOTTOM, 30)
# -----
# 新建一个 标签 返回 标签 索引号
# label_id = IPS200PRO.label_create(x, y, width, height, str, mode)
#     x            横向坐标    |    必要参数  标签  的 X 轴坐标
#     y            竖向坐标    |    必要参数  标签  的 Y 轴坐标
#     width        标签宽度    |    必要参数  标签  的宽度
#     height       标签高度    |    必要参数  标签  的高度
#     str          标签内容    |    可选参数  支持中文英文 UTF-8 格式
#     mode         显示模式    |    可选参数  IPS200PRO.LABEL_xxx
#                                     参数 xxx = (AUTO, DOT, SCROLL, SCROLL_CIRCULAR, CLIP)
#                                     对应 (自动, 省略, 来回滚动, 循环滚动, 裁切显示)
#     return       返回内容    |    正常情况下返回对应 标签 的索引
# -----
label_id1 = ips200pro.label_create(0, 0, 100, 30, "标签", IPS200PRO.LABEL_SCROLL)
# -----
# 通过对象调用 输出模块的 label 部分的使用帮助信息
# IPS200PRO.help_label ()
# -----
ips200pro.help_label ()
# -----
# 修改 标签 内容
# IPS200PRO.label_string(label_id, str)
#     label_id     标签索引    |    必要参数  标签  的索引号
#     str          标签内容    |    必要参数  支持中文英文 UTF-8 格式
# -----
ips200pro.label_string(label_id1, "这是一个标签")
# -----
# 修改 标签 显示模式
# IPS200PRO.label_mode(label_id, mode)
#     label_id     标签索引    |    必要参数  标签  的索引号
#     mode         显示模式    |    必要参数  IPS200PRO.LABEL_xxx
#                                     参数 xxx = (AUTO, DOT, SCROLL, SCROLL_CIRCULAR, CLIP)
#                                     对应 (自动, 省略, 来回滚动, 循环滚动, 裁切显示)
# -----
ips200pro.label_mode(label_id1, IPS200PRO.DOT)
# -----
# 修改组件字体
```



```
# IPS200PRO.set_font(widgets_id, font)
#     widgets_id  组件索引    |    必要参数  组件  的索引号
#     font        字体大小    |    必要参数  IPS200PRO.FONT_SIZE_xxx
#                                     参数 xxx = (12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 32, 34, 36, 40)
#                                     需要注意 仅 16、20、24 号字体支持中文显示
# -----
ips200pro.set_font(label_id1, IPS200PRO.FONT_SIZE_24)
# -----
# 修改组件颜色
# IPS200PRO.set_color(widgets_id, type, color)
#     widgets_id  组件索引    |    必要参数  组件  的索引号
#     type        设置选项    |    必要参数  指定要设置的是什么的颜色 IPS200PRO.COLOR_xxx
#                                     LABEL 组件可用参数 xxx = (FOREGROUND, BACKGROUND, BORDER)
#                                     对应 (前景色, 背景色, 组件边线颜色)
#     color       颜色数值    |    必要参数  RGB565 格式的颜色
# -----
ips200pro.set_color(label_id1, IPS200PRO.COLOR_FOREGROUND, 0xF800)
ips200pro.set_color(label_id1, IPS200PRO.COLOR_BACKGROUND, 0x07C0)
ips200pro.set_color(label_id1, IPS200PRO.COLOR_BORDER      , 0x001F)
# -----
# 修改组件位置
# IPS200PRO.set_position(widgets_id, x, y)
#     widgets_id  组件索引    |    必要参数  组件  的索引号
#     x          横向坐标    |    必要参数  组件  的 X 轴坐标
#     y          竖向坐标    |    必要参数  组件  的 Y 轴坐标
# -----
ips200pro.set_position(label_id1, 20, 120)
# -----
# 设置组件隐藏显示
# IPS200PRO.set_hidden(widgets_id, enable)
#     widgets_id  组件索引    |    必要参数  组件  的索引号
#     enable      隐藏使能    |    必要参数  True 隐藏 False 取消隐藏
# -----
ips200pro.set_hidden(label_id1, True)
# -----
# 修改组件父对象 可用于将组件切换到其它页面 或者给组件嵌套组件
# IPS200PRO.set_parent(widgets_id1, widgets_id2)
#     widgets_id1 组件索引    |    必要参数  组件  的索引号 子对象
#     widgets_id2 组件索引    |    必要参数  组件  的索引号 父对象
# -----
ips200pro.set_parent(label_id1, page_id1)

# 每个组件都是一个对象 对象之间可以设置继承关系 但只进行位置和范围的继承
# 设置父对象 就是将本对象 widgets_id1 设置与目标对象 widgets_id2 的继承关系
# 本对象就是子对象 目标对象就是父对象 此时子对象就只能在父对象的组件范围内操作
# 子对象的位置坐标就相对于父对象进行偏移 超出父对象大小范围的部分将不会显示
# 直接操作父对象的位置 子对象会随着父对象一同移动 可简单看做附着效果
```

3.4.10.3.table 表格相关接口

表格组件，可以用来组合显示信息，也可以设置为其它组件的子对象来组合使用。例如设置为波形图的子对象，用来显示各波形的标签或者数值等。


```

from seekfree import IPS200PRO
ips200pro = IPS200PRO(IPS200PRO.TITLE_BOTTOM, 30)
# -----
# 新建一个 表格 返回 表格 索引号
# table_id = IPS200PRO.table_create(x, y, row, col)
#     x           横向坐标   |   必要参数 表格 的 X 轴坐标
#     y           竖向坐标   |   必要参数 表格 的 Y 轴坐标
#     row         表格行数   |   必要参数 表格行数
#     col         表格列数   |   必要参数 表格列数
#     return      返回内容   |   正常情况下返回对应 表格 的索引
# -----
table_id1 = ips200pro.table_create( 0, 0, 8, 2)
# -----
# 通过对象调用 输出模块的 table 部分的使用帮助信息
# IPS200PRO.help_table()
# -----
ips200pro.help_table ()
# -----
# 修改 表格 内容
# IPS200PRO.table_string(table_id, row, col, str)
#     table_id     表格索引   |   必要参数 表格 的索引号
#     row          表格行数   |   必要参数 表格 行数
#     col          表格列数   |   必要参数 表格 列数
#     str          标签内容   |   必要参数 支持中文英文 UTF-8 格式
# -----
ips200pro.table_string(table_id1, 1, 1, "选项")
# -----
# 修改 表格 列宽
# IPS200PRO.table_col_width(table_id, col, width)
#     table_id     表格索引   |   必要参数 表格 的索引号
#     col          表格列数   |   必要参数 表格 列数
#     width        列宽数值   |   必要参数 支持中文英文 UTF-8 格式
# -----
ips200pro.table_col_width(table_id1, 1, 60)
# -----
# 选中 表格 单元格或整行整列 需要注意的是 选中效果是所有表格组件共享 因此同一时间只能有一个表格设置选中
# IPS200PRO.table_select(table_id, row, col)
#     table_id     表格索引   |   必要参数 表格 的索引号
#     row          表格行数   |   必要参数 表格 行数 为 0 时 为选择整列 超过表格行数时 取消选择
#     col          表格列数   |   必要参数 表格 列数 为 0 时 为选择整行 超过表格列数时 取消选择
# -----
ips200pro.table_select(table_id1, 2, 1)
# -----
# 修改组件字体
# IPS200PRO.set_font(widgets_id, font)
#     widgets_id   组件索引   |   必要参数 组件 的索引号
#     font         字体大小   |   必要参数 IPS200PRO.FONT_SIZE_xxx
#                                     参数 xxx = (12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 32, 34, 36, 40)
#                                     需要注意 仅 16、20、24 号字体支持中文显示
# -----
ips200pro.set_font(table_id1, IPS200PRO.FONT_SIZE_24)
# -----
# 修改组件颜色
# IPS200PRO.set_color(widgets_id, type, color)
#     widgets_id   组件索引   |   必要参数 组件 的索引号
#     type         设置选项   |   必要参数 指定要设置的是什么的颜色 IPS200PRO.COLOR_xxx
#                                     TABLE 组件可用参数 xxx = (FOREGROUND, BACKGROUND, BORDER, TABLE_SELECTED_BG)
#                                     对应 (前景色, 背景色, 组件边线颜色, 表格选中后的颜色)
# -----

```

```
# color 颜色数值 | 必要参数 RGB565 格式的颜色
# -----
ips200pro.set_color(table_id1, IPS200PRO.COLOR_FOREGROUND, 0x0000)
ips200pro.set_color(table_id1, IPS200PRO.COLOR_BACKGROUND, 0xFFFF)
ips200pro.set_color(table_id1, IPS200PRO.COLOR_BORDER, 0x001F)
ips200pro.set_color(table_id1, IPS200PRO.COLOR_TABLE_SELECTED_BG, 0x0000)
# -----
# 修改组件位置
# IPS200PRO.set_position(widgets_id, x, y)
# widgets_id 组件索引 | 必要参数 组件 的索引号
# x 横向坐标 | 必要参数 组件 的 X 轴坐标
# y 竖向坐标 | 必要参数 组件 的 Y 轴坐标
# -----
ips200pro.set_position(table_id1, 20, 120)
# -----
# 设置组件隐藏显示
# IPS200PRO.set_hidden(widgets_id, enable)
# widgets_id 组件索引 | 必要参数 组件 的索引号
# enable 隐藏使能 | 必要参数 True 隐藏 False 取消隐藏
# -----
ips200pro.set_hidden(table_id1, True)
# -----
# 修改组件父对象 可用于将组件切换到其它页面 或者给组件嵌套组件
# IPS200PRO.set_parent(widgets_id1, widgets_id2)
# widgets_id1 组件索引 | 必要参数 组件 的索引号 子对象
# widgets_id2 组件索引 | 必要参数 组件 的索引号 父对象
# -----
ips200pro.set_parent(table_id1, page_id1)

# 每个组件都是一个对象 对象之间可以设置继承关系 但只进行位置和范围的继承
# 设置父对象 就是将本对象 widgets_id1 设置与目标对象 widgets_id2 的继承关系
# 本对象就是子对象 目标对象就是父对象 此时子对象就只能在父对象的组件范围内操作
# 子对象的位置坐标就相对于父对象进行偏移 超出父对象大小范围的部分将不会显示
# 直接操作父对象的位置 子对象会随着父对象一同移动 可简单看做附着效果
```

3.4.10.4.meter 仪表相关接口

仪表组件，有角度指示和速度指示器两种风格。

```
from seekfree import IPS200PRO
ips200pro = IPS200PRO(IPS200PRO.TITLE_BOTTOM, 30)
# -----
# 新建一个 仪表 返回 仪表 索引号
# meter_id = IPS200PRO.meter_create(x, y, diameter, style)
# x 横向坐标 | 必要参数 仪表 的 X 轴坐标
# y 竖向坐标 | 必要参数 仪表 的 Y 轴坐标
# diameter 仪表直径 | 必要参数 最小不能小于 1
# style 仪表样式 | 必要参数 IPS200PRO.(METER_ANGLE, METER_SPEED) 圆盘角度和速度样式
# return 返回内容 | 正常情况下返回对应 仪表 的索引
# -----
meter_id1 = ips200pro.meter_create( 0, 0, 240, IPS200PRO.METER_SPEED)
# -----
# 通过对象调用 输出模块的 meter 部分的使用帮助信息
# IPS200PRO.help_meter()
# -----
```

```

ips200pro.help_meter ()
# -----
# 修改 仪表 数据
# IPS200PRO.meter_value(meter_id, value)
#     meter_id    仪表索引    |    必要参数 仪表 的索引号
#     value       仪表数值    |    必要参数 仪表 对应数值 METER_ANGLE 范围为 [0, 360] METER_SPEED 范围为 [0, 100]
# -----
ips200pro.meter_value(meter_id1, 50)
# -----
# 修改组件字体
# IPS200PRO.set_font(widgets_id, font)
#     widgets_id  组件索引    |    必要参数 组件 的索引号
#     font        字体大小    |    必要参数 IPS200PRO.FONT_SIZE_xxx
#                                     参数 xxx = (12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 32, 34, 36, 40)
# -----
ips200pro.set_font(meter_id1, IPS200PRO.FONT_SIZE_12)
# -----
# 修改组件颜色
# IPS200PRO.set_color(widgets_id, type, color)
#     widgets_id  组件索引    |    必要参数 组件 的索引号
#     type        设置选项    |    必要参数 指定要设置的是什么的颜色 IPS200PRO.COLOR_xxx
#                                     METER 可用 xxx = (FOREGROUND, BACKGROUND, BORDER, MRTER_INDICATOR, MRTER_TICKS)
#                                     对应 (前景色, 背景色, 仪表指针颜色, 仪表刻度颜色)
#     color       颜色数值    |    必要参数 RGB565 格式的颜色
# -----
ips200pro.set_color(meter_id1, IPS200PRO.COLOR_FOREGROUND, ips200pro.rgb888_to_rgb565(0xFF0000))
ips200pro.set_color(meter_id1, IPS200PRO.COLOR_BACKGROUND, ips200pro.rgb888_to_rgb565(0x000000))
ips200pro.set_color(meter_id1, IPS200PRO.COLOR_BORDER, ips200pro.rgb888_to_rgb565(0x000000))
ips200pro.set_color(meter_id1, IPS200PRO.COLOR_MRTER_INDICATOR, ips200pro.rgb888_to_rgb565(0xFFFF))
ips200pro.set_color(meter_id1, IPS200PRO.COLOR_MRTER_TICKS, ips200pro.rgb888_to_rgb565(0x66CCFF))
# -----
# 修改组件位置
# IPS200PRO.set_position(widgets_id, x, y)
#     widgets_id  组件索引    |    必要参数 组件 的索引号
#     x           横向坐标    |    必要参数 组件 的 X 轴坐标
#     y           竖向坐标    |    必要参数 组件 的 Y 轴坐标
# -----
ips200pro.set_position(meter_id1, 20, 120)
# -----
# 设置组件隐藏显示
# IPS200PRO.set_hidden(widgets_id, enable)
#     widgets_id  组件索引    |    必要参数 组件 的索引号
#     enable      隐藏使能    |    必要参数 True 隐藏 False 取消隐藏
# -----
ips200pro.set_hidden(meter_id1, True)
# -----
# 修改组件父对象 可用于将组件切换到其它页面 或者给组件嵌套组件
# IPS200PRO.set_parent(widgets_id1, widgets_id2)
#     widgets_id1 组件索引    |    必要参数 组件 的索引号 子对象
#     widgets_id2 组件索引    |    必要参数 组件 的索引号 父对象
# -----
ips200pro.set_parent(meter_id1, page_id1)

# 每个组件都是一个对象 对象之间可以设置继承关系 但只进行位置和范围的继承
# 设置父对象 就是将本对象 widgets_id1 设置与目标对象 widgets_id2 的继承关系
# 本对象就是子对象 目标对象就是父对象 此时子对象就只能在父对象的组件范围内操作
# 子对象的位置坐标就相对于父对象进行偏移 超出父对象大小范围的部分将不会显示
# 直接操作父对象的位置 子对象会随着父对象一同移动 可简单看做附着效果

```

3.4.10.5.clock 时钟相关接口

时钟组件，有圆形表盘和数字两种风格。

```
from seekfree import IPS200PRO
ips200pro = IPS200PRO(IPS200PRO.TITLE_BOTTOM, 30)
# -----
# 新建一个 时钟 返回 时钟 索引号
# clock_id = IPS200PRO.clock_create(x, y, size, type)
#     x            横向坐标    |    必要参数 组件 的 X 轴坐标
#     y            竖向坐标    |    必要参数 组件 的 Y 轴坐标
#     size          时钟尺寸    |    必要参数 模拟模式下是直径 [80, 240] 数字模式下是字体 IPS200PRO.FONT_SIZE_xxx
#                               参数 xxx = (12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 32, 34, 36, 40)
#     type          时钟类型    |    必要参数 IPS200PRO.(CLOCK_DIGITAL, CLOCK_ANALOG) 数字时钟 圆形时钟
#     return        返回内容    |    正常情况下返回对应 时钟 的索引
# -----
clock_id = ips200pro.clock_create(40, 0, 160, IPS200PRO.CLOCK_ANALOG)
# -----
# 通过对象调用 输出模块的 clock 部分的使用帮助信息
# IPS200PRO.help_clock()
# -----
ips200pro.help_clock()
# -----
# 修改组件字体
# IPS200PRO.set_font(widgets_id, font)
#     widgets_id  组件索引    |    必要参数 组件 的索引号
#     font        字体大小    |    必要参数 IPS200PRO.FONT_SIZE_xxx
#                               参数 xxx = (12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 32, 34, 36, 40)
# -----
ips200pro.set_font(clock_id, IPS200PRO.FONT_SIZE_12)
# -----
# 修改组件颜色
# IPS200PRO.set_color(widgets_id, type, color)
#     widgets_id  组件索引    |    必要参数 组件 的索引号
#     type        设置选项    |    必要参数 指定要设置的是什么的颜色 IPS200PRO.COLOR_xxx
#                               CLOCK 组件可用参数 xxx = (FOREGROUND, BACKGROUND, BORDER,
#                               CLOCK_HOUR, CLOCK_MINUTE, CLOCK_SECOND, CLOCK_TICKS)
#                               对应 (前景色, 背景色, 组件边线颜色,
#                               圆形时钟时针颜色, 圆形时钟分针颜色, 圆形时钟秒针颜色, 圆形时钟刻度颜色)
#     color       颜色数值    |    必要参数 RGB565 格式的颜色
# -----
ips200pro.set_color(clock_id, IPS200PRO.COLOR_FOREGROUND, 0x0000)
ips200pro.set_color(clock_id, IPS200PRO.COLOR_BACKGROUND, 0x0000)
ips200pro.set_color(clock_id, IPS200PRO.COLOR_BORDER, 0x0000)
# 修改 圆形时钟时针颜色 圆形时钟分针颜色 圆形时钟秒针颜色 圆形时钟刻度颜色
# 数字时钟则没有以下四个属性设置
ips200pro.set_color(clock_id, IPS200PRO.COLOR_CLOCK_HOUR, 0xFFFF)
ips200pro.set_color(clock_id, IPS200PRO.COLOR_CLOCK_MINUTE, 0xFFFF)
ips200pro.set_color(clock_id, IPS200PRO.COLOR_CLOCK_SECOND, 0xFFFF)
ips200pro.set_color(clock_id, IPS200PRO.COLOR_CLOCK_TICKS, ips200pro.rgb888_to_rgb565(0x39C5BB))
# -----
# 修改组件位置
# IPS200PRO.set_position(widgets_id, x, y)
#     widgets_id  组件索引    |    必要参数 组件 的索引号
#     x            横向坐标    |    必要参数 组件 的 X 轴坐标
```

```
#      y          竖向坐标      |      必要参数 组件 的 Y 轴坐标
# -----
ips200pro.set_position(clock_id, 20, 120)
# -----
# 设置组件隐藏显示
# IPS200PRO.set_hidden(widgets_id, enable)
#      widgets_id  组件索引      |      必要参数 组件 的索引号
#      enable      隐藏使能      |      必要参数 True 隐藏 False 取消隐藏
# -----
ips200pro.set_hidden(clock_id, True)
# -----
# 修改组件父对象 可用于将组件切换到其它页面 或者给组件嵌套组件
# IPS200PRO.set_parent(widgets_id1, widgets_id2)
#      widgets_id1  组件索引      |      必要参数 组件 的索引号 子对象
#      widgets_id2  组件索引      |      必要参数 组件 的索引号 父对象
# -----
ips200pro.set_parent(clock_id, page_id1)

# 每个组件都是一个对象 对象之间可以设置继承关系 但只进行位置和范围的继承
# 设置父对象 就是将本对象 widgets_id1 设置与目标对象 widgets_id2 的继承关系
# 本对象就是子对象 目标对象就是父对象 此时子对象就只能在父对象的组件范围内操作
# 子对象的位置坐标就相对于父对象进行偏移 超出父对象大小范围的部分将不会显示
# 直接操作父对象的位置 子对象会随着父对象一同移动 可简单看做附着效果
```

3.4.10.6.progress_bar 进度条相关接口

进度组件，一个无字符显示的单纯组件，可设置滑块位置。

```
from seekfree import IPS200PRO
ips200pro = IPS200PRO(IPS200PRO.TITLE_BOTTOM, 30)
# -----
# 新建一个 进度条 返回 进度条 索引号
# progress_bar_id = IPS200PRO.progress_bar_create(x, y, width, height)
#      x          横向坐标      |      必要参数 进度条 的 X 轴坐标
#      y          竖向坐标      |      必要参数 进度条 的 Y 轴坐标
#      width      组件宽度      |      必要参数 进度条 的宽度
#      height     组件高度      |      必要参数 进度条 的高度
#      return     返回内容      |      正常情况下返回对应 进度条 的索引
# -----
progress_bar_id1 = ips200pro.progress_bar_create( 20, 100, 200, 30)
# -----
# 通过对象调用 输出模块的 progress_bar 部分的使用帮助信息
# IPS200PRO.help_progress_bar()
# -----
ips200pro.help_progress_bar()
# -----
# 修改 进度条 数值 通过设置起始和停止位置来设置进度条的位置
# IPS200PRO.progress_bar_value(progress_bar_id, start_value, end_value)
#      progress_bar_id  进度条值      |      必要参数 进度条 的索引号
#      start_value      起始数值      |      必要参数 正整数
#      end_value        结束数值      |      必要参数 正整数
# -----
ips200pro.progress_bar_value(progress_bar_id1, 50, 75)
# -----
# 修改组件颜色
```

```
# IPS200PRO.set_color(widgets_id, type, color)
#     widgets_id  组件索引    |    必要参数  组件  的索引号
#     type        设置选项    |    必要参数  指定要设置的是什么的颜色 IPS200PRO.COLOR_xxx
#                     PROGRESS BAR 组件可用参数 xxx = (FOREGROUND, BACKGROUND)
#                     对应 (前景色, 背景色)
#     color        颜色数值    |    必要参数  RGB565 格式的颜色
# -----
ips200pro.set_color(progress_bar_id1, IPS200PRO.COLOR_FOREGROUND, 0xF800)
ips200pro.set_color(progress_bar_id1, IPS200PRO.COLOR_BACKGROUND, 0x0000)
# -----
# 修改组件位置
# IPS200PRO.set_position(widgets_id, x, y)
#     widgets_id  组件索引    |    必要参数  组件  的索引号
#     x           横向坐标    |    必要参数  组件  的 X 轴坐标
#     y           竖向坐标    |    必要参数  组件  的 Y 轴坐标
# -----
ips200pro.set_position(progress_bar_id1, 20, 120)
# -----
# 设置组件隐藏显示
# IPS200PRO.set_hidden(widgets_id, enable)
#     widgets_id  组件索引    |    必要参数  组件  的索引号
#     enable      隐藏使能    |    必要参数  True 隐藏 False 取消隐藏
# -----
ips200pro.set_hidden(progress_bar_id1, True)
# -----
# 修改组件父对象 可用于将组件切换到其它页面 或者给组件嵌套组件
# IPS200PRO.set_parent(widgets_id1, widgets_id2)
#     widgets_id1  组件索引    |    必要参数  组件  的索引号 子对象
#     widgets_id2  组件索引    |    必要参数  组件  的索引号 父对象
# -----
ips200pro.set_parent(progress_bar_id1, page_id1)

# 每个组件都是一个对象 对象之间可以设置继承关系 但只进行位置和范围的继承
# 设置父对象 就是将本对象 widgets_id1 设置与目标对象 widgets_id2 的继承关系
# 本对象就是子对象 目标对象就是父对象 此时子对象就只能在父对象的组件范围内操作
# 子对象的位置坐标就相对于父对象进行偏移 超出父对象大小范围的部分将不会显示
# 直接操作父对象的位置 子对象会随着父对象一同移动 可简单看做附着效果
```

3.4.10.7.calendar 日历相关接口

日历组件，显示一个日历，可显示范围是 1970-2099。

```
from seekfree import IPS200PRO
ips200pro = IPS200PRO(IPS200PRO.TITLE_BOTTOM, 30)
# -----
# 新建一个 日历 返回 日历 索引号
# calendar_id = IPS200PRO.calendar_create(x, y, width, height)
#     x           横向坐标    |    必要参数  日历 的 X 轴坐标
#     y           竖向坐标    |    必要参数  日历 的 Y 轴坐标
#     width       日历宽度    |    必要参数  日历 的宽度
#     height      日历高度    |    必要参数  日历 的高度
#     return      返回内容    |    正常情况下返回对应 日历 的索引
# -----
calendar_id = ips200pro.calendar_create( 0, 0, 240, 240)
# -----
```



```
# 通过对象调用 输出模块的 calendar 部分的使用帮助信息
# IPS200PRO.help_calendar()
# -----
ips200pro.help_calendar()
# -----
# 日历 定位到指定年月显示
# IPS200PRO.calendar_locate(year, month, mode)
#     year      定位年份    | 必要参数 年份范围是 [1970, 2099]
#     month     定位月份    | 必要参数 月份范围是 [1, 12]
#     mode      显示模式    | 必要参数 IPS200PRO.(CALENDAR_CHINESE, CALENDAR_ENGLISH) 中英文模式
# -----
ips200pro.calendar_locate(1970, 1, IPS200PRO.CALENDAR_CHINESE)
# -----
# 修改组件字体
# IPS200PRO.set_font(widgets_id, font)
#     widgets_id  组件索引    | 必要参数 组件 的索引号
#     font        字体大小    | 必要参数 IPS200PRO.FONT_SIZE_xxx
#                                     参数 xxx = (12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 32, 34, 36, 40)
#                                     需要注意 仅 16、20、24 号字体支持中文显示
# -----
ips200pro.set_font(calendar_id, IPS200PRO.FONT_SIZE_20)
# -----
# 修改组件颜色
# IPS200PRO.set_color(widgets_id, type, color)
#     widgets_id  组件索引    | 必要参数 组件 的索引号
#     type        设置选项    | 必要参数 指定要设置的是什么的颜色 IPS200PRO.COLOR_xxx
#                                     CALENDAR 组件可用参数 xxx = (FOREGROUND, BACKGROUND, BORDER,
#                                     CALENDAR_YEAR, CALENDAR_WEEK, CALENDAR_TODAY)
#                                     对应 (前景色, 背景色, 组件边线颜色, 年份字体颜色, 月份字体颜色, 今日日期框选颜色)
#     color       颜色数值    | 必要参数 RGB565 格式的颜色
# -----
ips200pro.set_color(calendar_id, IPS200PRO.COLOR_FOREGROUND, ips200pro.rgb888_to_rgb565(0x3B54D0))
ips200pro.set_color(calendar_id, IPS200PRO.COLOR_BACKGROUND, ips200pro.rgb888_to_rgb565(0xDEFF4F))
ips200pro.set_color(calendar_id, IPS200PRO.COLOR_BORDER, ips200pro.rgb888_to_rgb565(0x408EAF))
ips200pro.set_color(calendar_id, IPS200PRO.COLOR_CALENDAR_YEAR, ips200pro.rgb888_to_rgb565(0x39C5BB))
ips200pro.set_color(calendar_id, IPS200PRO.COLOR_CALENDAR_WEEK, ips200pro.rgb888_to_rgb565(0x39C5BB))
ips200pro.set_color(calendar_id, IPS200PRO.COLOR_CALENDAR_TODAY, ips200pro.rgb888_to_rgb565(0x66CCFF))
# -----
# 修改组件位置
# IPS200PRO.set_position(widgets_id, x, y)
#     widgets_id  组件索引    | 必要参数 组件 的索引号
#     x           横向坐标    | 必要参数 组件 的 X 轴坐标
#     y           竖向坐标    | 必要参数 组件 的 Y 轴坐标
# -----
ips200pro.set_position(calendar_id, 20, 120)
# -----
# 设置组件隐藏显示
# IPS200PRO.set_hidden(widgets_id, enable)
#     widgets_id  组件索引    | 必要参数 组件 的索引号
#     enable      隐藏使能    | 必要参数 True 隐藏 False 取消隐藏
# -----
ips200pro.set_hidden(calendar_id, True)
# -----
# 修改组件父对象 可用于将组件切换到其它页面 或者给组件嵌套组件
# IPS200PRO.set_parent(widgets_id1, widgets_id2)
#     widgets_id1 组件索引    | 必要参数 组件 的索引号 子对象
#     widgets_id2 组件索引    | 必要参数 组件 的索引号 父对象
# -----
```

```
ips200pro.set_parent(calendar_id, page_id1)
```

```
# 每个组件都是一个对象 对象之间可以设置继承关系 但只进行位置和范围的继承
# 设置父对象 就是将本对象 widgets_id1 设置与目标对象 widgets_id2 的继承关系
# 本对象就是子对象 目标对象就是父对象 此时子对象就只能在父对象的组件范围内操作
# 子对象的位置坐标就相对于父对象进行偏移 超出父对象大小范围的部分将不会显示
# 直接操作父对象的位置 子对象会随着父对象一同移动 可简单看做附着效果
```

3.4.10.8.waveform 波形图相关接口

波形图组件，可以用来显示数组数据波形，一个波形图中可以显示 5 条数据波形。

```
from seekfree import IPS200PRO
ips200pro = IPS200PRO(IPS200PRO.TITLE_BOTTOM, 30)
# -----
# 新建一个 波形图 返回 波形图 索引号
# waveform_id = IPS200PRO.waveform_create(x, y, width, height)
#     x            横向坐标    |    必要参数 波形图 的 X 轴坐标
#     y            竖向坐标    |    必要参数 波形图 的 Y 轴坐标
#     width        组件宽度    |    必要参数 波形图 的宽度
#     height       组件高度    |    必要参数 波形图 的高度
#     return       返回内容    |    正常情况下返回对应 波形图 的索引
# -----
waveform_id1 = ips200pro.waveform_create( 0, 0, 240, 100)
# -----
# 通过对象调用 输出模块的 waveform 部分的使用帮助信息
# IPS200PRO.help_waveform()
# -----
ips200pro.help_waveform()
# -----
# 向 波形图 指定 ID 的波形添加数据 可以输入字节数组 或添加单个数据
# IPS200PRO.waveform_value(waveform_id, line_id, data_list, color = 0xF800)
#     waveform_id 波形索引    |    必要参数 波形图 的索引号
#     line_id     线条索引    |    必要参数 线条 的索引号 一个波形图中最多五条线 范围为 [1, 5]
#     data_list   数据列表    |    必要参数 数据 传入一个整数数组 数值范围是 [0, 100]
#     color       线条颜色    |    可选参数 线条 颜色 RGB565 格式的颜色 默认红色
# IPS200PRO.waveform_value(waveform_id, line_id, data, color = 0xF800)
#     waveform_id 波形索引    |    必要参数 波形图 的索引号
#     line_id     线条索引    |    必要参数 线条 的索引号 一个波形图中最多五条线 范围为 [1, 5]
#     data        数据数值    |    必要参数 数据 传入一个整数数值 数值范围是 [0, 100]
#     color       线条颜色    |    可选参数 线条 颜色 RGB565 格式的颜色 默认红色
# -----
from array import *
wave_sin_1 = array('h', [0] * (250))
ips200pro.waveform_value(waveform_id1, 1, wave_sin_1, 0xF800)
ips200pro.waveform_value(waveform_id1, 1, 75)
# -----
# 修改指定 波形图 中 线条 的显示状态
# IPS200PRO.waveform_line(waveform_id, line_id, enable = True)
#     waveform_id 波形索引    |    必要参数 波形图 的索引号
#     line_id     线条索引    |    必要参数 线条 的索引号 一个波形图中最多五条线 范围为 [1, 5]
#     enable      线条状态    |    可选参数 默认为 True 显示线条 可输入 False 隐藏显示
# -----
ips200pro.waveform_line(waveform_id1, 1, False)
ips200pro.waveform_line(waveform_id1, 1)
```



```
# -----
# 设置 波形图 中 线条 的显示模式
# IPS200PRO.waveform_mode(waveform_id, connect)
#     waveform_id 波形索引 | 必要参数 波形图 的索引号
#     connect      线条模式 | 可选参数 True 点之间连接折线显示 False 点之间不连接散点显示
# -----
ips200pro.waveform_mode(waveform_id1, True)
# -----
# 清空 波形图 中所有 线条 数据
# IPS200PRO.waveform_clear(waveform_id)
#     waveform_id 波形索引 | 必要参数 波形图 的索引号
# -----
ips200pro.waveform_clear(waveform_id1)
# -----
# 修改组件颜色
# IPS200PRO.set_color(widgets_id, type, color)
#     widgets_id 组件索引 | 必要参数 组件 的索引号
#     type        设置选项 | 必要参数 指定要设置的是什么的颜色 IPS200PRO.COLOR_xxx
#                   WAVEFORM 组件可用参数 xxx = (BACKGROUND)
#                   对应 (背景色)
#     color       颜色数值 | 必要参数 RGB565 格式的颜色
# -----
ips200pro.set_color(waveform_id1, IPS200PRO.COLOR_BACKGROUND, 0x0000)
# -----
# 修改组件位置
# IPS200PRO.set_position(widgets_id, x, y)
#     widgets_id 组件索引 | 必要参数 组件 的索引号
#     x          横向坐标 | 必要参数 组件 的 X 轴坐标
#     y          竖向坐标 | 必要参数 组件 的 Y 轴坐标
# -----
ips200pro.set_position(waveform_id1, 20, 120)
# -----
# 设置组件隐藏显示
# IPS200PRO.set_hidden(widgets_id, enable)
#     widgets_id 组件索引 | 必要参数 组件 的索引号
#     enable     隐藏使能 | 必要参数 True 隐藏 False 取消隐藏
# -----
ips200pro.set_hidden(waveform_id1, True)
# -----
# 修改组件父对象 可用于将组件切换到其它页面 或者给组件嵌套组件
# IPS200PRO.set_parent(widgets_id1, widgets_id2)
#     widgets_id1 组件索引 | 必要参数 组件 的索引号 子对象
#     widgets_id2 组件索引 | 必要参数 组件 的索引号 父对象
# -----
ips200pro.set_parent(waveform_id1, page_id1)

# 每个组件都是一个对象 对象之间可以设置继承关系 但只进行位置和范围的继承
# 设置父对象 就是将本对象 widgets_id1 设置与目标对象 widgets_id2 的继承关系
# 本对象就是子对象 目标对象就是父对象 此时子对象就只能在父对象的组件范围内操作
# 子对象的位置坐标就相对于父对象进行偏移 超出父对象大小范围的部分将不会显示
# 直接操作父对象的位置 子对象会随着父对象一同移动 可简单看做附着效果
```

3.4.10.9.container 容器相关接口

容器组件，单纯的一个容器，可以用来作为其它组件的父对象或者用来遮挡、同色掩盖。

```
from seekfree import IPS200PRO
ips200pro = IPS200PRO(IPS200PRO.TITLE_BOTTOM, 30)
# -----
# 新建一个 容器 返回 容器 索引号
# container_id = IPS200PRO.container_create(x, y, width, height)
#     x            横向坐标    |    必要参数 容器 的 X 轴坐标
#     y            竖向坐标    |    必要参数 容器 的 Y 轴坐标
#     width        容器宽度    |    必要参数 容器 的宽度
#     height       容器高度    |    必要参数 容器 的高度
#     return       返回内容    |    正常情况下返回对应 容器 的索引
# -----
container_id1 = ips200pro.container_create( 0, 0, 240, 240)
# -----
# 通过对象调用 输出模块的 container 部分的使用帮助信息
# IPS200PRO.help_container()
# -----
ips200pro.help_container()
# -----
# 修改 容器 样式
# IPS200PRO.container_radius(container_id, border_width, radius)
#     container_id 容器索引    |    必要参数 容器 的索引号
#     border_width 边线宽度    |    必要参数 容器 边线宽度
#     radius       圆角半径    |    必要参数 容器 圆角半径
# -----
ips200pro.container_radius(container_id1, 0, 0)
# -----
# 修改组件颜色
# IPS200PRO.set_color(widgets_id, type, color)
#     widgets_id  组件索引    |    必要参数 组件 的索引号
#     type        设置选项    |    必要参数 指定要设置的是什么的颜色 IPS200PRO.COLOR_xxx
#                 CONTAINER 组件可用参数 xxx = (BACKGROUND, BORDER)
#                 对应 (背景色, 组件边线颜色)
#     color       颜色数值    |    必要参数 RGB565 格式的颜色
# -----
ips200pro.set_color(container_id1, IPS200PRO.COLOR_BACKGROUND, ips200pro.rgb888_to_rgb565(0xACACAC))
ips200pro.set_color(container_id1, IPS200PRO.COLOR_BORDER, ips200pro.rgb888_to_rgb565(0x000000))
# -----
# 修改组件位置
# IPS200PRO.set_position(widgets_id, x, y)
#     widgets_id  组件索引    |    必要参数 组件 的索引号
#     x           横向坐标    |    必要参数 组件 的 X 轴坐标
#     y           竖向坐标    |    必要参数 组件 的 Y 轴坐标
# -----
ips200pro.set_position(container_id1, 20, 120)
# -----
# 设置组件隐藏显示
# IPS200PRO.set_hidden(widgets_id, enable)
#     widgets_id  组件索引    |    必要参数 组件 的索引号
#     enable      隐藏使能    |    必要参数 True 隐藏 False 取消隐藏
# -----
ips200pro.set_hidden(container_id1, True)
```

```
# -----  
# 修改组件父对象 可用于将组件切换到其它页面 或者给组件嵌套组件  
# IPS200PRO.set_parent(widgets_id1, widgets_id2)  
#     widgets_id1 组件索引 | 必要参数 组件 的索引号 子对象  
#     widgets_id2 组件索引 | 必要参数 组件 的索引号 父对象  
# -----  
ips200pro.set_parent(container_id1, page_id1)  
  
# 每个组件都是一个对象 对象之间可以设置继承关系 但只进行位置和范围的继承  
# 设置父对象 就是将本对象 widgets_id1 设置与目标对象 widgets_id2 的继承关系  
# 本对象就是子对象 目标对象就是父对象 此时子对象就只能在父对象的组件范围内操作  
# 子对象的位置坐标就相对于父对象进行偏移 超出父对象大小范围的部分将不会显示  
# 直接操作父对象的位置 子对象会随着父对象一同移动 可简单看做附着效果
```

4. 多文件调用与类定义

虽然使用 Python 编写程序可以用较少的代码量完成任务，但渡过起步阶段后引入更多的算法、决策必然会导致代码量增大，此时就有分文件管理的需求了。

4.1. 多文件加载

RT1021 允许多文件加载，当通过 import 包含其他文件时，它是将文件加载到内存作为一个对象来使用，因此使用 from name import * 的方式引用文件时，如果不同文件内有同样名称的函数，就会被重载覆盖掉：

```
# func.py 文件
def func1(x):
    return (x+1)

# func1.py 文件
def func1(x):
    return (x+2)
```

使用 REPL 测试如下：

```
Shell x
MPY: soft reboot
MicroPython v1.20.0 on 2025-02-11; RT1021 MicroPython by NXP & SeekFree with CoreBoard-144Pin V2.1.0
Type "help()" for more information.
>>> from func import *
>>> func1(1)
2
>>> from func1 import *
>>> func1(1)
3
>>> |
```

因此如果多个文件中包含同样的 class、function 的话，尽量使用 import name 的方式然后通过对象调用：

```
Shell x
MPY: soft reboot
MicroPython v1.20.0 on 2025-02-11; RT1021 MicroPython by NXP & SeekFree with CoreBoard-144Pin V2.1.0
Type "help()" for more information.
>>> import func
>>> import func1
>>> func.func1(1)
2
>>> func1.func1(1)
3
>>> |
```

4.2.多文件变量作用域

基于这个文件加载的特性，在多文件使用全局变量时，建议通过 `import name`，使用 `name.value` 的方式访问全局变量，否则容易导致因作用域不清晰而产生的数据错误。例如我们使用如下两个文件测试变量作用域：

func.py 文件：

```
data = 10

def func (x):
    global data
    return(x + data)
```

user_main.py 文件：

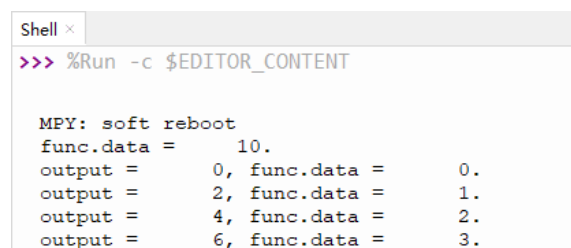
```
from machine import *
import func
import gc, time

num = 0
led = Pin('C4' , Pin.OUT, pull = Pin.PULL_UP_47K, value = True)

def test(x):
    func.data = x
    return func.func(x)

while True:
    time.sleep_ms(500)
    led.toggle()
    print("output = {:>6d}, func.data = {:>6d}.".format(test(num), func.data))
    num = num + 1
    gc.collect()
```

运行 `user_main` 文件就会看到如下效果，这里就由于是通过对象访问，因此传递的变量就是正确的值，输出值是 `func.py` 文件中 `data` 变量的两倍（传入值赋值给了 `data`）：



```
Shell x
>>> %Run -c $EDITOR_CONTENT

MPY: soft reboot
func.data =      10.
output =      0, func.data =      0.
output =      2, func.data =      1.
output =      4, func.data =      2.
output =      6, func.data =      3.
```

而如果使用 `from name import *` 的方式引用文件，可能会导致 `user_main` 文件中的函数无法调用到 `func` 文件中的变量，使用 `global` 时就会额外自行新建一个变量：

`main.py` 文件：

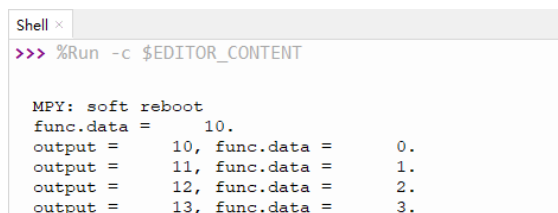
```
from machine import *
from func import *
import gc, time

num = 0
led = Pin('C4', Pin.OUT, pull = Pin.PULL_UP_47K, value = True)
print("func.data = {:>6d}.".format(data)) # 此时输出的是 func.py 文件中 data 变量的值

def test(x):
    global data # 设想它要引用 func.py 文件中 data 变量
    data = x # 但实际它是新建了一个变量替代掉了 data 这个标签
    return func(x) # 在这之后的代码 data 标签将不再访问到 func.py 文件中 data 变量

while True:
    time.sleep_ms(500)
    led.toggle()
    print("output = {:>6d}, func.data = {:>6d}.".format(test(num), data))
    num = num + 1
    gc.collect()
```

而此时 `main` 中会额外新建 `global` 的 `data` 变量覆盖了“`data`”这个标签，导致计算结果并非我们设想的那样（`func` 文件中的 `data` 一直为 10 没有改变，改变的是 `main` 中的 `data` 标签的变量值），并且我们也无法再访问到 `func` 文件中的 `data` 变量。



```
Shell x
>>> %Run -c $EDITOR_CONTENT

MPY: soft reboot
func.data =      10.
output =      10, func.data =      0.
output =      11, func.data =      1.
output =      12, func.data =      2.
output =      13, func.data =      3.
```

4.3.类的应用

除了《4.2.多文件变量作用域》的变量跨文件调用方式，还可以使用自拟定类的方式，通过传递对象的方式来传递数据。例如电机控制、专项控制需要用到 `PID` 算法，此时可以将 `PID` 算法定义一个类，编写成单独的文件 `pid_func.py`：

```
class pid_class:
    def __init__(self, kp = 0, ki = 0, kd = 0, ei = 0, ei_max = 0, output_max = 0):
        self.kp = kp
    ...
    # 代码略

    def pid_standard_integral (self, error):
        self.e1 = self.e0
    ...
    # 代码略

    def pid_standard_incremental (self, error):
        self.e1 = self.e0
    ...
    # 代码略
```

这样直接通过 from name import * 的方式引用文件直接获得类定义，跨文件也可以调用：

func.py 文件：

```
from pid_func import *

def func (x):
    print(x.kp)
```

func1.py 文件：

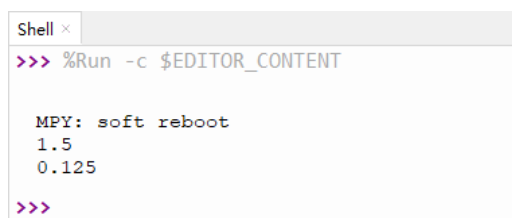
```
def func1 (x):
    print(x.ki)
```

main.py 文件：

```
from func import *
from func1 import *

gyro_pid = pid_class(kp = 1.5, ki = 0.125)
func(gyro_pid)
func1(gyro_pid)
```

运行得到结果：



```
Shell x
>>> %Run -c $EDITOR_CONTENT

MPY: soft reboot
1.5
0.125
>>>
```

虽然在 func1.py 文件中并未引用 pid_func 文件，但是由于它们都实际在 main.py 文件引用，而 main.py 通过 func.py 获得了 pid_func 的类定义，所以它依旧可以运行。也就是实际上 main.py 将各文件引用过来后在同一个作用域（main）生效。

5.文档版本

版本号	日期	作者	内容变更
V1.0	2024/3/8	云猫	初始版本。
V1.1	2024/3/9	云猫	修改 Thonny 配图与说明。
V1.2	2024/3/11	云猫	增加多文件调用说明
V1.3	2024/3/12	云猫	根据反馈修改描述，并增加 Ticker 启动处理的描述
V1.4	2024/3/15	云猫	新增版本描述，新增固件启动说明并配图
V1.5	2024/4/9	云猫	新增 DL1B 支持描述 修改原有描述 新增脱机运行详述
V1.6	2024/12/9	云猫	修改部分描述，修复部分错误
V2.0	2025/2/12	云猫	统一 V2.1.0 固件版本新说明书初版。
V3.0	2026/4/15	云猫	固件全面更新，所有版本统一到一个接口说明